

PROYECTO DE FIN DE CICLO

TÍTULO DEL PROYECTO:

Automatización del despliegue y administración
de servicios en contenedores mediante Bash, Ansible y
Docker
IJ Systems Automation



Administración de Sistemas Informáticos en Red (ASIR)

AUTORES	Ian Gutiérrez Guertsen Jaime Ferrari Zamorano
TUTORA DEL PROYECTO	Marina Villarán Bermejo
COORDINADORA	Beatriz Herrezuelo Sánchez
CENTRO ESCOLAR	I.E.S. Rosa Chacel 
CURSO	2025/2026
FECHA DE ENTREGA	Mayo de 2026

ÍNDICE GENERAL

1	Introducción	5
1.1	Abstract (English version).....	6
2	Alcance del proyecto.....	7
3	Estudio de viabilidad.....	8
3.1	Estado actual del sistema.....	8
3.2	Requisitos del cliente	8
3.3	Posibles soluciones	9
3.4	Solución elegida.....	9
3.5	Planificación temporal de las tareas del proyecto.....	9
3.6	Planificación de los recursos a utilizar	10
3.7	Viabilidad económica	10
3.7.1	Estimación económica fuera de AWS Academy	11
4	Análisis	12
4.1	Requisitos funcionales	12
4.2	Requisitos no funcionales.....	12
5	Diseño	14
5.1	Estructura de la aplicación	14
5.2	Componentes del sistema.....	15
5.3	Arquitectura de la red	15
5.4	Herramientas	16
6	Implementación.....	17
6.1	Entorno de implementación.....	24
6.2	Estructuras técnicas	26
6.3	Carga de datos.....	26
6.4	Ficheros de configuración actualizados	31
6.5	Configuraciones realizadas en el sistema.....	31
6.6	Implementaciones de código realizadas	32
6.7	Caso práctico completo de despliegue y validación en AWS	44
7	Pruebas.....	45
7.1	Casos de prueba.....	45
8	Explotación	52
8.1	Planificación.....	52
8.2	Preparación para el cambio	53
8.3	Plan de formación	53
8.4	Implantación propiamente dicha.....	53
8.5	Pruebas de implantación	53
8.6	Paso a un entorno productivo	53
9	Definición de procedimientos de control y evaluación	55
10	Conclusiones.....	56
11	Glosario.....	56
12	Bibliografía	58

13	Anexos	59
13.1	Manual básico de usuario.....	59
13.1.1	Acceso al servidor.....	59
13.1.2	Servicios disponibles	60
13.1.3	Puesta en marcha del entorno	60
13.1.4	Comprobación del estado	60
13.1.5	Copia de seguridad.....	60
13.1.6	Restauración.....	61
13.1.7	Validación de MariaDB.....	61
13.1.8	Recomendaciones de uso	62
13.2	Planificación operativa resumida.....	62
13.2.1	Fase 1. Preparación de infraestructura	62
13.2.2	Fase 2. Acceso y preparación del host.....	62
13.2.3	Fase 3. Despliegue del repositorio	62
13.2.4	Fase 4. Levantamiento de servicios.....	63
13.2.5	Fase 5. Validación funcional.....	63
13.2.6	Fase 6. Explotación básica y mantenimiento.....	63
13.3	Procedimiento resumido de despliegue y validación	63
13.4	Registro de incidencias y troubleshooting.....	64
13.4.1	Incidencia 1. Error de autenticación al subir cambios a GitHub	64
13.4.2	Incidencia 2. Permisos insuficientes sobre el socket de Docker	64
13.4.3	Incidencia 3. Acceso incorrecto a MariaDB	64
13.4.4	Incidencia 4. Validación de persistencia de MariaDB.....	65
13.4.5	Incidencia 5. Ajuste de puertos y acceso externo	65
13.5	Documentación complementaria del repositorio	66
13.6	Índice de anexos y documentación externa	66
13.7	Cierre del apartado de anexos.....	67

ÍNDICE DE TABLAS

Tabla 3.1	Planificación detallada del proyecto con hitos, entregables y contingencia.	10
Tabla 3.2	Estimación económica orientativa fuera de AWS Academy.	11
Tabla 4.1	Matriz de trazabilidad entre requisitos, implementación y pruebas.	13
Tabla 4.2	Relación entre objetivos SMART, criterios de aceptación y evidencias de validación.	14
Tabla 5.1	Componentes del sistema.....	15
Tabla 6.1	Inventario técnico del entorno (elementos, uso, versiones y comprobación).	25
Tabla 6.2	Estructuras técnicas.....	26
Tabla 6.3	Ficheros de configuración.	31
Tabla 6.4	Implementaciones de código.	32
Tabla 6.5	Puertos, exposición y justificación de seguridad.	33
Tabla 6.6	Desarrollo del caso práctico completo.	44
Tabla 7.1	Casos de prueba.	46
Tabla 7.2	Relación de evidencias utilizadas en la validación.	47
Tabla 7.3	Comparativa entre proceso manual y proceso automatizado.	47
Tabla 9.1	Plantilla de registro de incidencias y control de cambios.	55
Tabla 13.1	Análisis complementario de riesgos técnicos y medidas de mitigación.	66
Tabla 13.2	Índice de anexos y documentación externa.....	67

ÍNDICE DE FIGURAS

Figura 1.1 : Diagrama de flujo del proyecto.	7
Figura 5.1 : Arquitectura lógica del despliegue implementado.....	16
Figura 6.1 : Selección de Ubuntu Server LTS para la instancia EC2.....	18
Figura 6.2 : Selección del tipo de instancia EC2.	18
Figura 6.3 : Creación del par de claves SSH para la instancia EC2.	19
Figura 6.4 : Configuración de red y grupo de seguridad de la instancia EC2.....	19
Figura 6.5 : Reglas de entrada y almacenamiento configurados en la instancia EC2.	20
Figura 6.6 : Resumen de la configuración de la instancia EC2 antes del lanzamiento.....	20
Figura 6.7 : Reglas de entrada finales del grupo de seguridad de la instancia EC2.	21
Figura 6.8 : Instancia EC2 creada y en ejecución.....	22
Figura 6.9 : Instrucciones de conexión SSH a la instancia EC2.	22
Figura 6.10 : Conexión SSH correcta a la instancia EC2.	23
Figura 6.11 : Verificación del usuario y directorio inicial en la instancia EC2.	23
Figura 6.12 : Asignación de una IP elástica en Amazon EC2.....	23
Figura 6.13 : Asociación de la IP elástica a la instancia EC2.....	24
Figura 6.14 : Confirmación de la asociación de la IP elástica a la instancia EC2.....	24
Figura 6.15 : Variables de entorno definidas en el fichero .env.	27
Figura 6.16 : Comprobación de exclusión del fichero .env y uso de variables de entorno.	28
Figura 6.17 : Configuración de Prometheus para la recogida de métricas.	29
Figura 6.18 : Parte del contenido HTML de prueba publicado mediante Nginx.	29
Figura 6.19 : Validación de la base de datos MariaDB desde consola.....	30
Figura 6.20 : Distribución funcional de los contenedores del proyecto.....	32
Figura 6.21 : Resumen visual de los scripts Bash del proyecto.....	34
Figura 6.22 : Script Bash para la copia de seguridad de MariaDB.....	35
Figura 6.23 : Script Bash de instalación y preparación del servidor.	35
Figura 6.24 : Script Bash de comprobación del entorno desplegado.	36
Figura 6.25 : Script Bash de restauración de copias de seguridad de MariaDB.....	36
Figura 6.26 : Inventario de Ansible utilizado para la configuración local.	37
Figura 6.27 : Variables globales utilizadas por Ansible.	37
Figura 6.28 : Playbook de Ansible para la preparación y despliegue del servidor.....	38
Figura 6.29 : Primera parte del fichero docker-compose.yml.....	39
Figura 6.30 : Definición final de servicios, redes y volúmenes en docker-compose.yml.	40
Figura 6.31 : Configuración del proxy inverso en nginx.conf.....	41
Figura 6.32 : Despliegue correcto de la pila de contenedores con Docker Compose.....	42
Figura 6.33 : Imagen del Repositorio GitHub.....	43
Figura 6.34 : Estructura de directorios del repositorio del proyecto.	43
Figura 7.1 : Validación del servicio web Nginx desde navegador.	48
Figura 7.2 : Validación de objetivos activos en Prometheus.	49
Figura 7.3 : Supervisión del servicio HTTP en Uptime Kuma.	49
Figura 7.4 : Visualización de contenedores Docker en cAdvisor.....	49
Figura 7.5 : Validación limpia de inserción y consulta de datos en MariaDB.	50
Figura 7.6 : Ejecución correcta del script de copia de seguridad.	50
Figura 7.7 : Eliminación de la tabla de prueba antes de la restauración.....	50
Figura 7.8 : Restauración correcta de MariaDB mediante restore.sh.	51
Figura 7.9 : Validación de datos restaurados en MariaDB.	51
Figura 7.10 : Trazabilidad del repositorio y estructura final del proyecto.	52
Figura 13.1 : Configuración de la tarea programada de backup mediante cron.	61
Figura 13.2 : Ejecución del backup automático y fichero de copia generado.....	61

Licencia del TFG:

La presente memoria del proyecto “IJ Systems Automation” está bajo una licencia Creative Commons Reconocimiento–NoComercial–SinObraDerivada 4.0 Internacional (CC BY-NC-ND 4.0).

Se permite la descarga, consulta y distribución de esta memoria siempre que se reconozca la autoría de sus autores. No se permite su uso con fines comerciales ni la creación de obras derivadas.

Autores: Ian Gutiérrez Guertsen, Jaime Ferrari Zamorano

Para ver una copia de esta licencia, visite:

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

1 Introducción

En este documento se presenta el proyecto de fin de ciclo “Automatización del despliegue y administración de servicios en contenedores mediante Bash, Ansible y Docker”, desarrollado para el CFGS de Administración de Sistemas Informáticos en Red.

La propuesta surgió de una necesidad habitual en entornos de sistemas: reducir el tiempo invertido en instalaciones repetitivas, minimizar errores humanos y disponer de un procedimiento reproducible para desplegar servicios en servidores Linux. Para ello, se combinó el uso de scripts Bash, playbooks de Ansible y contenedores Docker sobre un entorno Ubuntu, utilizando AWS como plataforma de despliegue.¹

La elección del tema no es casual. Las prácticas de automatización y los enfoques DevOps se han convertido en una competencia transversal exigida en perfiles de administración de sistemas, operaciones IT y soporte a infraestructura cloud. Dominar herramientas como Bash, Ansible y Docker no es un conocimiento aislado, sino el núcleo de un perfil técnico moderno. El proyecto pretende, por tanto, salvar la distancia entre los contenidos del ciclo formativo y la realidad operativa que el alumnado encontrará en su primer entorno profesional.

El objetivo general consistió en diseñar una solución mantenible y documentada que permitiera preparar el servidor, desplegar servicios, actualizar configuraciones y ejecutar tareas de administración de forma homogénea, obteniendo como resultado una base técnica reutilizable en futuros entornos de laboratorio o de pequeña empresa. Como hilo conductor, se definieron cinco objetivos específicos, medibles y verificables (OBJ-01 a OBJ-05): automatizar la preparación del servidor Ubuntu en una única ejecución, desplegar la pila completa de servicios en menos de tres minutos, validar la publicación web desde Internet, comprobar la persistencia y recuperación de MariaDB mediante copias de seguridad, y mantener la trazabilidad técnica en un repositorio GitHub. Cada objetivo dispone de un criterio de aceptación concreto y de una evidencia asociada, lo que permite verificar de forma objetiva el grado de cumplimiento alcanzado al cierre del trabajo.

¹ Documentación oficial de Amazon EC2 (Amazon Web Services). Disponible en: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>. Recurso utilizado para la creación y configuración de la instancia base del proyecto.

Final Project Licence:

This project report "IJ Systems Automation" is licensed under a Creative Commons Attribution–NonCommercial–NoDerivatives 4.0 International licence (CC BY-NC-ND 4.0).

Downloading, consulting and distributing this report is permitted provided that the authorship of its authors is acknowledged. Its use for commercial purposes and the creation of derivative works are not permitted.

Authors: Ian Gutiérrez Guertsen, Jaime Ferrari Zamorano

To view a copy of this licence, visit:

<https://creativecommons.org/licenses/by-nc-nd/4.0/>

1.1 Abstract (*English version*)

This document presents the end-of-cycle project "Automation of the deployment and administration of containerised services using Bash, Ansible and Docker", developed for the Higher Vocational Training Cycle in Network Computer Systems Administration (ASIR).

The proposal arose from a common need in systems environments: reducing the time spent on repetitive installations, minimising human error and providing a reproducible procedure for deploying services on Linux servers. To achieve this, the project combined Bash scripts, Ansible playbooks and Docker containers on an Ubuntu environment, using AWS as the deployment platform.

The topic was not chosen by chance. Automation practices and DevOps approaches have become a cross-cutting competency required in systems administration, IT operations and cloud infrastructure support roles. Mastering tools such as Bash, Ansible and Docker is not an isolated skill but the core of a modern technical profile. The project therefore aims to bridge the gap between the contents of the training cycle and the operational reality that students will encounter in their first professional environment.

The general objective was to design a maintainable and well-documented solution capable of preparing the server, deploying services, updating configurations and performing administration tasks in a consistent way, resulting in a reusable technical foundation for future laboratory or small-business environments. As a guiding thread, five specific, measurable and verifiable objectives were defined (OBJ-01 to OBJ-05): automating the preparation of the Ubuntu server in a single execution, deploying the full service stack in under three minutes, validating web publishing from the Internet, verifying MariaDB persistence and recovery through backups, and maintaining technical traceability in a GitHub repository. Each objective has a specific acceptance criterion and associated evidence, allowing the degree of completion achieved at the end of the work to be verified objectively.

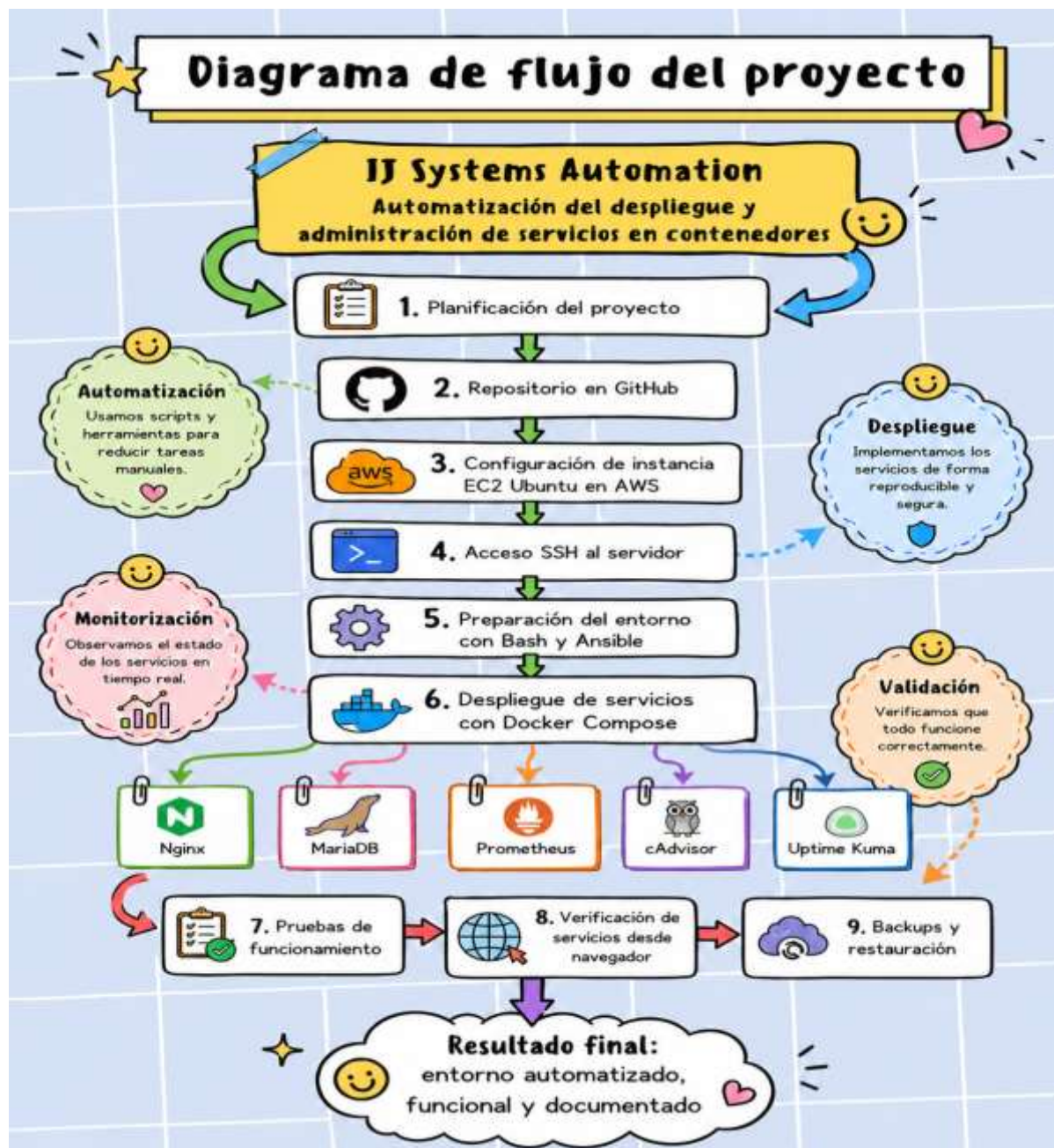


Figura 1.1 : Diagrama de flujo del proyecto.

Fuente: elaboración propia.

2 Alcance del proyecto

El alcance del proyecto comprendió el análisis, diseño e implantación de un entorno automatizado para la puesta en marcha de servicios en contenedores Docker, administrados de manera centralizada. Se contemplaron tanto la fase de preparación del sistema como la explotación básica de los servicios desplegados.

Dentro del alcance se incluyeron el aprovisionamiento inicial del servidor Ubuntu, la instalación automatizada de dependencias, la definición del repositorio de trabajo, la elaboración de scripts Bash, la construcción de inventarios y playbooks de Ansible, la orquestación de servicios mediante Docker Compose y la validación mediante pruebas técnicas.

Quedaron fuera del alcance la implantación corporativa de alta disponibilidad, la orquestación avanzada con Kubernetes, el balanceo complejo entre varios nodos y la migración de datos reales de producción. El proyecto se orientó a un escenario realista de automatización, administrable por un equipo reducido.

Tareas realizadas:

- Preparación automática del servidor base.
- Despliegue reproducible de servicios en contenedores.
- Gestión centralizada mediante Ansible.
- Automatización de tareas operativas con Bash.
- Elaboración de documentación técnica, pruebas y procedimiento de explotación.

3 Estudio de viabilidad

Se analizó la viabilidad técnica, organizativa y económica del proyecto, tomando como referencia un escenario de pequeña empresa o laboratorio que necesitaba agilizar despliegues y reducir tareas manuales de administración.

3.1 Estado actual del sistema

En el estado inicial, la instalación de servicios se realizaba de forma manual, comando a comando, con tiempos de preparación variables y riesgo de generar diferencias entre servidores. Cada despliegue dependía en exceso de la persona que lo ejecutaba y de la documentación disponible en ese momento.

Esta situación provocaba problemas frecuentes, como configuraciones inconsistentes, dificultad para replicar entornos, mayor tiempo de recuperación ante incidencias y escasa trazabilidad de los cambios. Además, la actualización de servicios o la reinstalación del sistema obligaba a repetir procedimientos que podían automatizarse.

3.2 Requisitos del cliente

El cliente necesitaba una solución sencilla de mantener, fácilmente reproducible y apoyada en tecnologías ampliamente utilizadas en la administración de sistemas. Para ello, se definieron los siguientes requisitos:

- Automatizar la preparación inicial del servidor.
- Permitir el despliegue de varios servicios sin repetir tareas manuales.
- Centralizar configuraciones y cambios en ficheros versionados.
- Facilitar copias de seguridad, comprobaciones de estado y actualización de servicios.
- Reducir los tiempos de implantación y minimizar los errores humanos.
- Permitir una futura ampliación del proyecto sin necesidad de rehacer toda la infraestructura.

3.3 Posibles soluciones

Se consideraron varias alternativas técnicas antes de seleccionar la solución final:

- Administración manual con scripts aislados: solución rápida de implantar, pero poco escalable y difícil de mantener.
- Uso exclusivo de Docker Compose: simplifica el despliegue de contenedores, aunque no cubre por sí solo la configuración global del servidor.
- Uso de herramientas de orquestación avanzada como Kubernetes: opción potente, pero excesiva para el tamaño y los objetivos del proyecto.
- Combinación de Bash, Ansible y Docker: opción equilibrada, modular y alineada con los conocimientos adquiridos en el ciclo formativo.

3.4 Solución elegida

La opción más adecuada fue una arquitectura basada en Bash, Ansible y Docker sobre Ubuntu Linux, desplegada en una instancia de Amazon EC2. Bash se utilizó para tareas del sistema operativo y automatizaciones rápidas; Ansible permitió aplicar configuraciones de forma declarativa y repetible; y Docker Compose facilitó la definición y puesta en marcha coordinada de los servicios en contenedores (Amazon Web Services, s. f.-g; Ansible Community, s. f.-a; Docker Inc., s. f.-a).^{2 3}

La solución resultó adecuada por su equilibrio entre sencillez, coste contenido, capacidad de ampliación y valor formativo. Además, favoreció una estructura modular del proyecto, con repositorio versionado y separación clara entre scripts, inventarios, configuraciones y definición de servicios.

3.5 Planificación temporal de las tareas del proyecto

La planificación se estructuró en fases consecutivas, aunque parcialmente solapables. Se estimó una dedicación repartida entre el análisis, la preparación del entorno, la automatización, las pruebas y la documentación.

Detalle adicional de planificación, hitos y entregables:

Junto a la planificación inicial, se definieron dependencias entre fases, entregables verificables y un margen de contingencia. Así, cada fase no solo tenía una duración estimada, sino también un resultado concreto que validaba su cierre antes de pasar a la siguiente.

² Documentación oficial de Ansible (Ansible Community). Disponible en: <https://docs.ansible.com/projects/ansible/latest/index.html>. Referencia utilizada para la elaboración del inventario y los playbooks del proyecto.

³ Documentación oficial de Docker Compose (Docker Inc.). Disponible en: <https://docs.docker.com/compose/>. Referencia utilizada para la orquestación de los contenedores de la pila desplegada.

Fase	Dependencia principal	Hito de cierre	Entregable	Margen de contingencia	Duración estimada
Análisis	Inicio del proyecto	Alcance y requisitos aprobados	Listado de requisitos y alcance	Revisión con tutoría	1 semana
Diseño	Análisis cerrado	Arquitectura lógica definida	Diseño de repositorio, red y servicios	Ajuste de componentes	1 semana
Preparación	Diseño validado	Instancia accesible por SSH	EC2, Security Group e IP elástica	Corrección de acceso o puertos	1 semana
Automatización	Host preparado	Scripts y playbook ejecutables	instalar.sh, checks.sh, backup.sh, restore.sh y playbook.yml	Depuración de permisos	2 semanas
Contenedores	Automatización funcional	Pila levantada con Docker Compose	docker-compose.yml, redes y volúmenes	Ajuste de puertos y variables	1 semana
Pruebas	Servicios desplegados	Validación funcional completa	Casos CP-01 a CP-14 y evidencias	Repetición de pruebas fallidas	1 semana

Tabla 3.1 : Planificación detallada del proyecto con hitos, entregables y contingencia.

Fuente: elaboración propia.

3.6 Planificación de los recursos a utilizar

Se emplearon recursos asequibles y habituales en entornos académicos y de pequeña empresa:

- Equipo cliente para el desarrollo, las pruebas y la documentación.
- Instancia Amazon EC2 con Ubuntu Linux para la implantación del entorno (Amazon Web Services, s. f.-g).
- Repositorio GitHub para el control de versiones y la trazabilidad del código fuente (GitHub, s. f.).
- Conexión a Internet, claves SSH y herramientas de administración remota.
- Tiempo de tutoría para la revisión del avance y la validación técnica.

3.7 Viabilidad económica

Desde el punto de vista económico, el proyecto se estructuró con el objetivo de reducir al máximo el gasto. Dado que se trataba de un entorno de pruebas y aprendizaje, no se consideró necesario realizar una inversión económica elevada para validar la arquitectura propuesta. La finalidad del proyecto no fue desplegar una infraestructura de producción permanente, sino construir una base funcional que permitiera demostrar la viabilidad técnica de la solución en un entorno preparado para ello.

Para llevar a cabo la implantación, se dispuso de acceso al laboratorio de AWS Academy, facilitado por el profesorado del área de Servicios en Red y Computación en la Nube. Este acceso permitió utilizar recursos de AWS sin asumir un coste directo durante el desarrollo del proyecto. Además, al tratarse de un trabajo realizado por dos autores, se contó con dos cuentas de laboratorio independientes, con un saldo aproximado de 50 € por cuenta, lo que proporcionó un margen suficiente para las pruebas previstas.

No se previó superar dicho saldo, ya que el uso de la infraestructura se realizó de forma puntual y controlada, únicamente durante las fases de despliegue, validación y comprobación del entorno. En ningún caso se contempló un uso continuo de tipo 24/7, por lo que el consumo estimado se mantuvo bajo. En consecuencia, no fue necesario realizar ningún gasto adicional, y el proyecto pudo considerarse económicamente viable dentro de los recursos académicos disponibles.

3.7.1 Estimación económica fuera de AWS Academy

Aunque el desarrollo del proyecto se realizó utilizando los recursos proporcionados por AWS Academy, también se elaboró una estimación orientativa del coste que tendría mantener la misma infraestructura en una cuenta estándar de AWS sin créditos académicos. Esta estimación permitió comprobar si la solución seguiría siendo viable económicamente en un escenario real de laboratorio, pruebas o pequeña empresa.

Para realizar el cálculo, se tomó como referencia una instancia ligera de tipo t3.micro con Linux/Unix en la región US East (N. Virginia), un volumen EBS gp3 de 20 GB y una dirección IPv4 pública asociada a la instancia. La estimación se basó en las páginas oficiales de precios de AWS para instancias EC2 bajo demanda, direcciones IPv4 públicas, almacenamiento EBS y en el uso de AWS Pricing Calculator como herramienta orientativa de planificación de costes (Amazon Web Services, s. f.-a; Amazon Web Services, s. f.-b; Amazon Web Services, s. f.-c; Amazon Web Services, s. f.-d).

Según la documentación de precios de Amazon Web Services, las instancias EC2 bajo demanda se facturan en función del tipo de instancia, sistema operativo, región y tiempo de uso; las direcciones IPv4 públicas pueden facturarse por hora cuando están en uso o reservadas; y el almacenamiento EBS gp3 se factura en función de los GB provisionados al mes (Amazon Web Services, s. f.-a; Amazon Web Services, s. f.-b; Amazon Web Services, s. f.-c).

Recurso	Estimación de uso	Cálculo orientativo	Coste aproximado
Instancia EC2 t3.micro Linux	720 horas/mes	0,0104 USD/h x 720 h	7,49 USD/mes
Dirección IPv4 pública / IP elástica	720 horas/mes	0,005 USD/h x 720 h	3,60 USD/mes
Volumen EBS gp3 de 20 GB	20 GB/mes	20 GB x 0,08 USD/GB-mes	1,60 USD/mes
Transferencia de datos	Uso bajo de pruebas	Tráfico reducido de validación	Coste muy reducido
Total estimado mensual	Uso continuo 24/7	-	12,69 USD/mes aprox.

Tabla 3.2 : Estimación económica orientativa fuera de AWS Academy.

Fuente: elaboración propia.

En caso de utilizar la instancia únicamente durante periodos puntuales de prueba, el coste sería considerablemente menor. Por ejemplo, si el entorno se mantuviera activo durante unas 50 horas totales para despliegue, validación y documentación, el coste aproximado de cómputo e IPv4 pública sería de unos 0,77 USD, sin contar el almacenamiento que pudiera mantenerse creado durante el mes. Este cálculo se realizó a partir de los precios horarios utilizados en la estimación de la instancia EC2 y de la dirección IPv4 pública (Amazon Web Services, s. f.-a; Amazon Web Services, s. f.-b).

Por tanto, incluso sin disponer de AWS Academy, el proyecto seguiría siendo económicamente viable. El coste mensual estimado para una infraestructura básica en funcionamiento continuo se situaría alrededor de 13 USD al mes, una cantidad asumible para un entorno académico, de laboratorio o de pequeña empresa.

4 Análisis

En esta fase se concretaron los requisitos que debía cumplir la solución para considerarse válida desde el punto de vista técnico y operativo.

4.1 Requisitos funcionales

El sistema debía cubrir, como mínimo, las siguientes funciones:

- Preparar un servidor Ubuntu para su uso como nodo de despliegue.
- Instalar de forma automatizada Docker, Docker Compose, Ansible y las utilidades necesarias.
- Permitir el despliegue y la parada de servicios en contenedores mediante órdenes controladas.
- Gestionar configuraciones, variables de entorno y ficheros de servicio desde un repositorio versionado.
- Ejecutar comprobaciones de estado, reinicios controlados y copias de seguridad básicas.
- Facilitar la actualización del entorno sin necesidad de realizar una reinstalación completa.
- Proporcionar documentación mínima de operación y recuperación.

4.2 Requisitos no funcionales

Además de las funciones anteriores, la solución debía satisfacer una serie de requisitos de calidad:

- Reproducibilidad del despliegue en un tiempo reducido.
- Mantenibilidad del código y de los ficheros de configuración.
- Legibilidad de scripts y playbooks mediante una estructura clara y comentarios.
- Seguridad básica en accesos, credenciales y exposición de servicios.
- Portabilidad razonable a otros entornos Linux compatibles.
- Escalabilidad limitada, pero suficiente para añadir nuevos servicios o roles.
- Trazabilidad de cambios mediante control de versiones.

Para asegurar que el proyecto no quedara únicamente como una descripción teórica, se relacionaron los requisitos principales con las partes implementadas y las pruebas que los validaron. Esta trazabilidad permitió comprobar de forma objetiva que cada necesidad definida en el análisis tenía una respuesta técnica y una evidencia de funcionamiento.

Requisito	Implementación asociada	Prueba relacionada	Estado
Preparar un servidor Ubuntu para despliegue	Instancia EC2, acceso SSH, instalación base y playbook de Ansible	CP-07	Correcto
Instalar Docker, Docker Compose, Ansible y utilidades	instalar.sh y playbook.yml	CP-07	Correcto
Desplegar servicios en contenedores	docker-compose.yml	CP-01 y CP-02	Correcto
Publicar servicio web	Contenedor Nginx y contenido estático de prueba	CP-03	Correcto
Supervisar disponibilidad	Uptime Kuma	CP-05	Correcto
Recoger métricas del host y contenedores	Prometheus, node-exporter y cAdvisor	CP-04 y CP-06	Correcto
Validar persistencia de datos	MariaDB con volumen Docker persistente	CP-08	Correcto
Realizar copia y recuperación básica	backup.sh y restore.sh	CP-09, CP-10 y CP-11	Correcto
Mantener trazabilidad del código	Repositorio GitHub con estructura versionada	CP-12	Correcto

Tabla 4.1 : Matriz de trazabilidad entre requisitos, implementación y pruebas.

Fuente: elaboración propia.

ID	Objetivo SMART	Criterio de aceptación	Evidencia
OBJ-01	Automatizar la preparación de un servidor Ubuntu para que Docker, Docker Compose y Ansible queden instalados en una única ejecución.	El script/playbook finaliza sin errores y docker --version, docker compose version y ansible --version devuelven resultado correcto.	CP-07
OBJ-02	Desplegar la pila de servicios mediante Docker Compose en menos de 3 minutos en un entorno de pruebas.	docker compose up -d deja los contenedores principales en ejecución.	CP-01 / CP-02

ID	Objetivo SMART	Criterio de aceptación	Evidencia
OBJ-03	Validar la disponibilidad del servicio web desde navegador mediante la IP pública de AWS.	La página de Nginx carga correctamente desde <code>http://<IP_PUBLICA_ELASTICA></code> .	CP-03
OBJ-04	Comprobar la persistencia y recuperación de MariaDB mediante backup y restore.	Los datos permanecen tras reinicio y pueden restaurarse desde un .sql.	CP-08 / CP-09 / CP-10 / CP-11
OBJ-05	Mantener trazabilidad técnica del proyecto mediante repositorio GitHub.	El código queda organizado, versionado y enlazado desde la memoria.	CP-12

Tabla 4.2 : Relación entre objetivos SMART, criterios de aceptación y evidencias de validación.

Fuente: elaboración propia.

5 Diseño

En esta fase se concretó el diseño técnico del proyecto a partir de los requisitos definidos en el análisis. La solución se planteó como una arquitectura modular, sencilla de mantener y orientada a despliegues reproducibles sobre Ubuntu.

El diseño se apoyó en Bash para automatizaciones locales, Ansible para la configuración centralizada y Docker Compose para la puesta en marcha coordinada de los servicios. De esta forma, se separaron claramente las tareas de administración, publicación, persistencia y monitorización.

5.1 Estructura de la aplicación

La estructura lógica del proyecto se organizó en un repositorio único, dividido por capas funcionales. Esta organización facilitó el mantenimiento, la reutilización del código y la futura ampliación del entorno sin necesidad de rehacer toda la solución.

- **scripts/**: contiene los scripts Bash destinados a tareas operativas, comprobaciones, copias de seguridad, restauración y automatizaciones rápidas.
- **ansible/**: contiene el inventario, los playbooks y las variables necesarias para configurar el servidor objetivo de forma centralizada.
- **docker/**: almacena el fichero docker-compose.yml, las variables de entorno, los volúmenes y las configuraciones específicas de los servicios.
- **monitoring/**: agrupa los ficheros de monitorización necesarios para la recogida y visualización de métricas.
- **docs/**: recopila documentación técnica, capturas, casos de prueba y evidencias de funcionamiento.

El flujo de trabajo consistió en preparar el host mediante Bash y Ansible, desplegar posteriormente la pila de servicios con Docker Compose y validar su estado mediante comprobaciones técnicas.

5.2 Componentes del sistema

Los componentes del sistema se seleccionaron para cubrir la publicación web, la persistencia de datos, la monitorización técnica y la supervisión de disponibilidad. La solución final quedó compuesta por **Nginx**, **MariaDB**, **Prometheus**, **node-exporter**, **cAdvisor** y **Uptime Kuma**.

Contenedor / componente	Función principal	Justificación en el proyecto
Nginx	Publicación web y punto de entrada HTTP.	Sirve una web de prueba y actúa como punto de entrada HTTP del proyecto.
MariaDB	Persistencia de datos	Permite demostrar volúmenes, variables de entorno y copias de seguridad básicas.
Prometheus	Recogida de métricas	Centraliza métricas técnicas del host y de los contenedores.
cAdvisor	Métricas de contenedores	Permite visualizar el uso de CPU, memoria, red y almacenamiento de los contenedores en ejecución.
Uptime Kuma	Supervisión de disponibilidad	Permite comprobar desde interfaz web si los servicios publicados responden correctamente.
node-exporter	Exportación de métricas del host	Permite que Prometheus recoja métricas de CPU, memoria, disco y red del servidor.

Tabla 5.1 : Componentes del sistema.

Fuente: elaboración propia.

Además de estos contenedores, el host Ubuntu mantuvo las funciones de control, conectividad SSH y ejecución de Ansible y Bash. De este modo, la lógica de administración quedó separada de la lógica de servicio.

Como componente técnico auxiliar, node-exporter se utilizó para exponer métricas del host a Prometheus. Su función no fue ofrecer un servicio final al usuario, sino completar la monitorización del servidor desde la red interna del entorno Docker.

5.3 Arquitectura de la red

La implantación se realizó sobre una instancia **Amazon EC2** con **Ubuntu Server**. El acceso administrativo se llevó a cabo mediante **SSH** desde el equipo de los autores, mientras que el despliegue de servicios se ejecutó localmente en el host mediante **Docker Compose** (Amazon Web Services, s. f.-g; Docker Inc., s. f.-a).

A nivel lógico, se empleó una red **frontend** para la exposición del servicio web y de las herramientas de validación, junto con una red interna de soporte para la monitorización y la comunicación entre servicios no expuestos directamente.

ARQUITECTURA LÓGICA PREVISTA DEL DESPLIEGUE

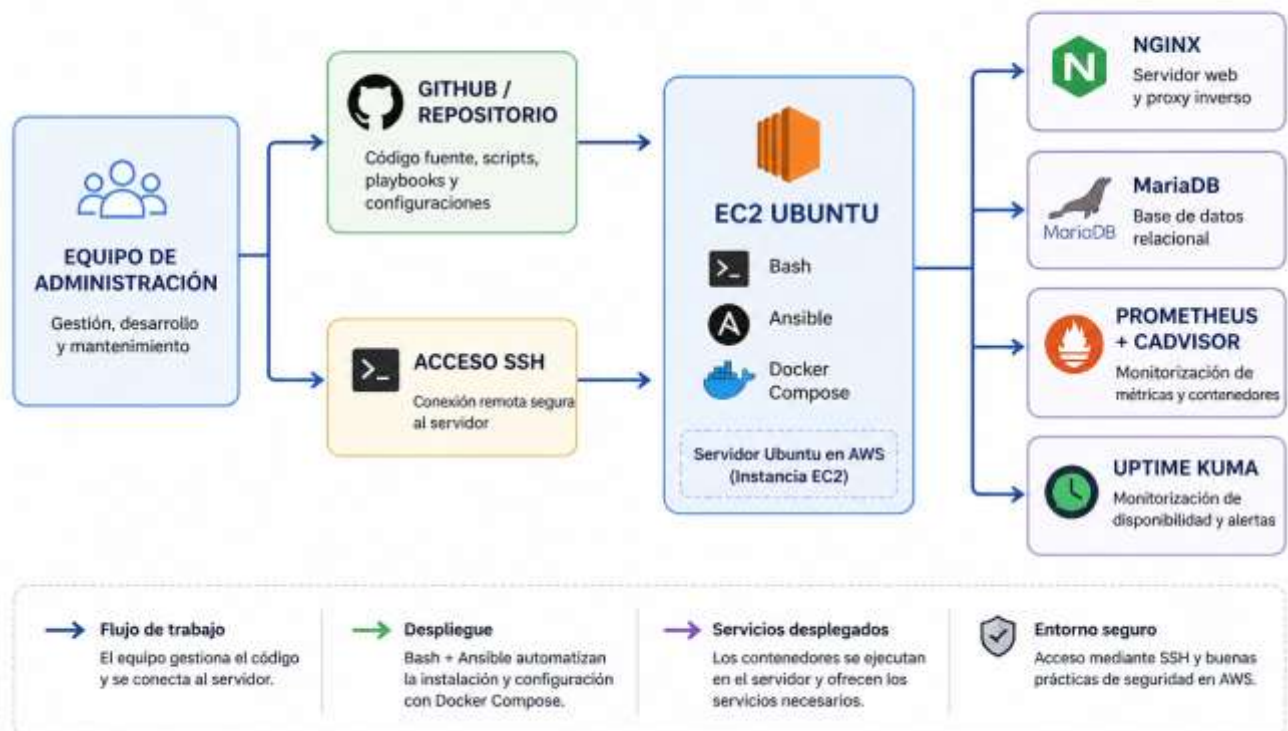


Figura 5.1 : Arquitectura lógica del despliegue implementado.

Fuente: elaboración propia.

5.4 Herramientas

Las herramientas seleccionadas se ajustaron al alcance del proyecto, ya que cada una cubrió una necesidad concreta dentro de la solución propuesta. Se utilizaron de forma complementaria, evitando solapamientos innecesarios y asignando a cada tecnología una función específica dentro del entorno.

- **Ubuntu Server LTS:** sistema operativo base de la instancia EC2, seleccionado por su estabilidad, soporte extendido y compatibilidad con herramientas de administración de sistemas (Ubuntu, s. f.).⁴
- **Amazon EC2:** plataforma de despliegue utilizada para ejecutar el entorno en la nube y validar el proyecto en una infraestructura externa al equipo local (Amazon Web Services, s. f.-g).
- **Bash:** automatización de tareas repetitivas del host, como comprobaciones, copias de seguridad, restauración y preparación inicial.

⁴ Documentación oficial de Ubuntu Server, sistema operativo base utilizado en el proyecto. Disponible en: <https://documentation.ubuntu.com/server/>.

- **Ansible:** configuración declarativa del servidor, instalación de dependencias y reducción de diferencias entre ejecuciones (Ansible Community, s. f.-a).
- **Docker y Docker Compose:** encapsulación y orquestación básica de los servicios, redes y volúmenes necesarios para el despliegue (Docker Inc., s. f.-a).
- **Nginx, MariaDB, Prometheus, Uptime Kuma, node-exporter y cAdvisor:** servicios desplegados para cubrir publicación web, persistencia de datos, monitorización del host, supervisión de disponibilidad y métricas de contenedores (Nginx, s. f.; Google, s. f.; Lam, s. f.; MariaDB plc, s. f.-a; Prometheus Authors, s. f.-d).
- **GitHub:** repositorio centralizado utilizado para mantener la trazabilidad del código, facilitar la revisión por parte del profesorado y conservar versiones funcionales del proyecto (GitHub, s. f.).

6 Implementación

Partiendo del diseño anterior, la implementación se realizó de forma incremental. Cada bloque se validó antes de continuar con el siguiente. En primer lugar, se aseguró el entorno base. Después, se automatizó la configuración del servidor. Por último, se desplegaron los contenedores y se comprobaron los ficheros asociados.

El apartado se centra en una solución de automatización y despliegue sobre infraestructura Linux, sin desarrollo de una aplicación con base de datos propia, por lo que las subfases descritas se ajustan a ese alcance.

Creación y configuración de la instancia en AWS:

Se seleccionó **Ubuntu Server LTS** como sistema operativo base por su estabilidad, compatibilidad con Docker Engine y uso habitual en entornos de servidor. A continuación, se asignó el nombre identificativo de la instancia (Amazon Web Services, s. f.-g; Docker Inc., s. f.-c):⁵

⁵ Guía oficial de instalación de Docker Engine sobre Ubuntu. Disponible en: <https://docs.docker.com/engine/install/ubuntu/>. Procedimiento seguido durante la fase de preparación del host.

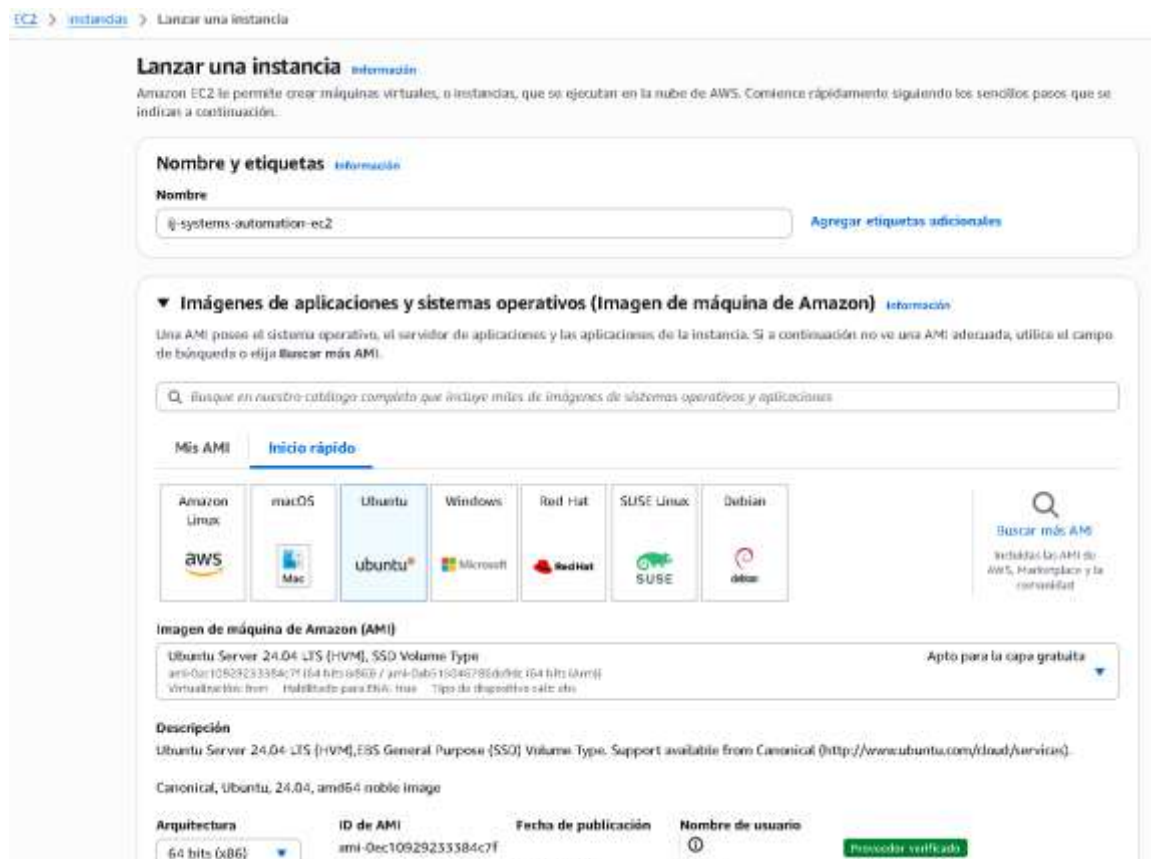


Figura 6.1 : Selección de Ubuntu Server LTS para la instancia EC2.

Fuente: elaboración propia.

Se mantuvo un tipo de instancia ligero y suficiente para el alcance del proyecto, priorizando el bajo consumo de recursos dentro del laboratorio de AWS Academy:



Figura 6.2 : Selección del tipo de instancia EC2.

Fuente: elaboración propia.

Se generó un par de claves para acceder a la instancia mediante **SSH**, evitando el uso de contraseña y reforzando la seguridad del acceso administrativo (Amazon Web Services, s. f.-f):

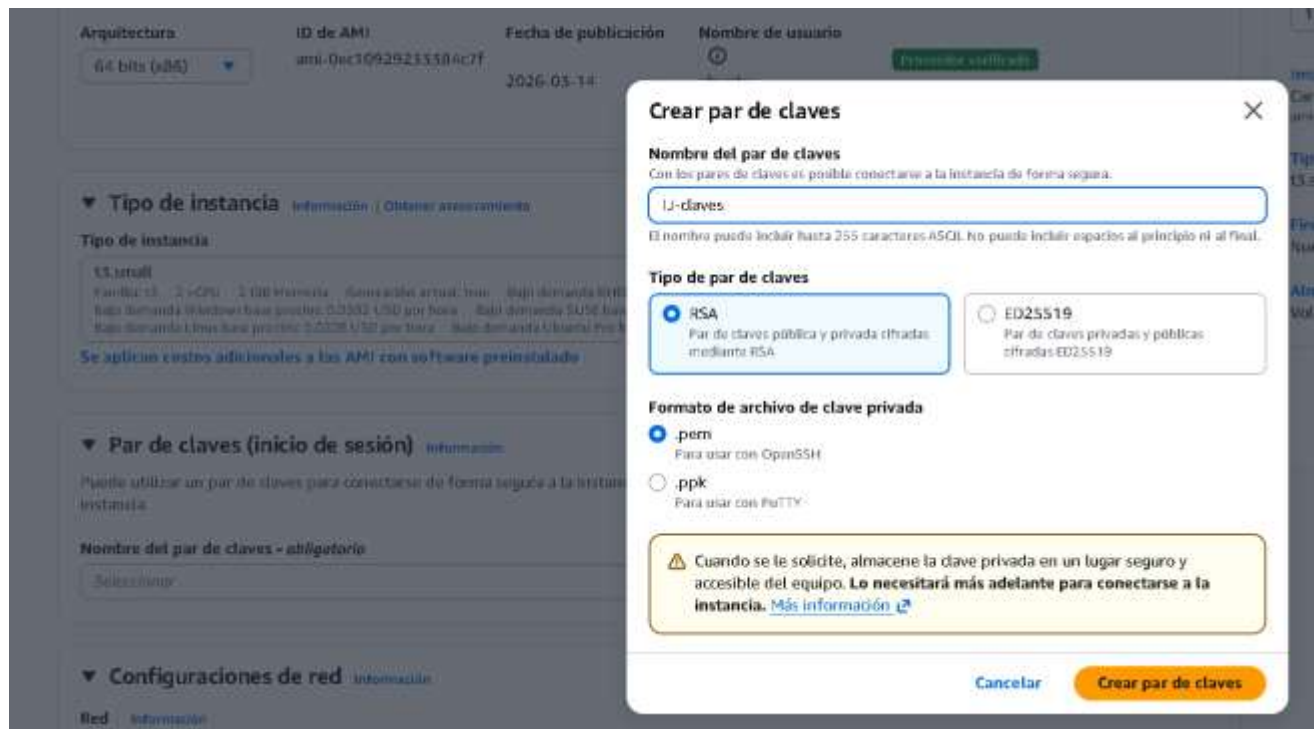


Figura 6.3 : Creación del par de claves SSH para la instancia EC2.

Fuente: elaboración propia.

Se creó un grupo de seguridad específico para controlar el tráfico de entrada hacia la instancia (Amazon Web Services, s. f.-c):⁶

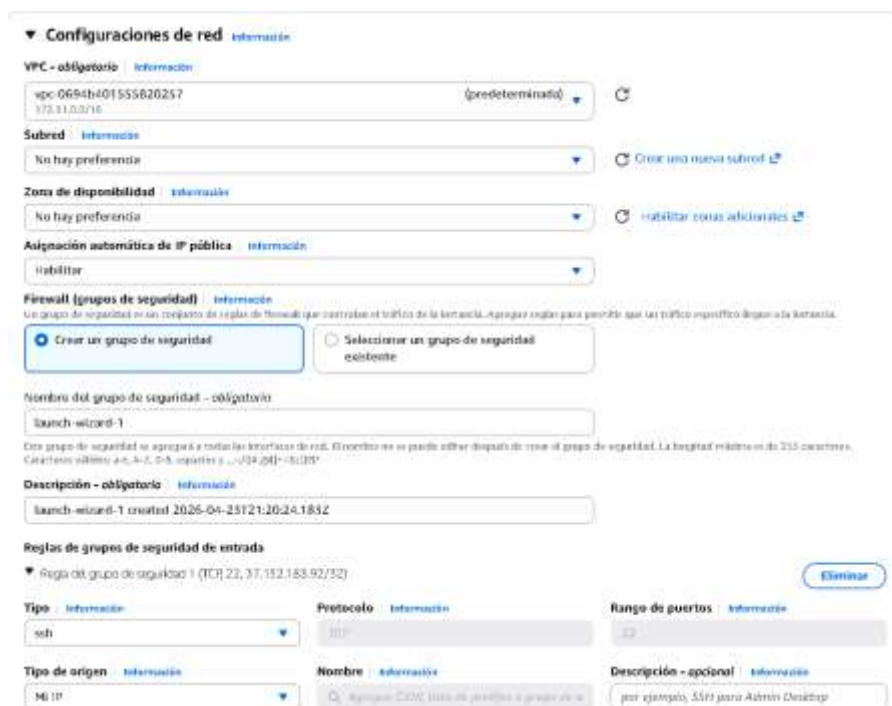


Figura 6.4 : Configuración de red y grupo de seguridad de la instancia EC2.

Fuente: elaboración propia.

⁶ Documentación oficial de grupos de seguridad de Amazon EC2. Disponible en: <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html>. Referencia utilizada para configurar las reglas de acceso a la instancia.

Se revisaron las reglas restantes y la configuración de almacenamiento antes de lanzar la instancia:

The screenshot displays the AWS Management Console configuration for an EC2 instance. It is divided into two main sections: 'Reglas de grupo de seguridad' (Security Group Rules) and 'Configurar almacenamiento' (Configure storage).

Reglas de grupo de seguridad:

- Regla 1:** Tipo: HTTPS, Protocolo: TCP, Rango de puertos: 443, Tipo de origen: Cualquier lugar, Origen: 0.0.0.0/0. Descripción: SSH para Admin Desktop.
- Regla 2:** Tipo: HTTP, Protocolo: TCP, Rango de puertos: 80, Tipo de origen: Cualquier lugar, Origen: 0.0.0.0/0. Descripción: SSH para Admin Desktop.

Configurar almacenamiento:

- 1x 20 GiB gp3 Volumen raíz, 3000 IOPS, No cifrado.

A note at the bottom states: 'La AMI seleccionada contiene volúmenes de almacenamiento de instancia; sin embargo, la instancia no permite dichos volúmenes. Por lo que no se podrá obtener'.

Figura 6.5 : Reglas de entrada y almacenamiento configurados en la instancia EC2.

Fuente: elaboración propia.

Resumen final de la configuración antes de crear la instancia:

The screenshot shows the 'Resumen' (Summary) page for an EC2 instance configuration. It provides a final overview of the settings before launching the instance.

Resumen

- Número de instancias:** 1
- Imagen de software (AMI):** Canonical, Ubuntu, 24.04, amd64... más información
ami-0ec1092923384c7f
- Tipo de servidor virtual (tipo de instancia):** t3.small
- Firewall (grupo de seguridad):** Nuevo grupo de seguridad
- Almacenamiento (volúmenes):** Volúmenes: 1 (20 GiB)

Buttons at the bottom: Cancelar, Lanzar instancia, and Código de versión preliminar.

Figura 6.6 : Resumen de la configuración de la instancia EC2 antes del lanzamiento.

Fuente: elaboración propia.

En esta fase se configuraron las reglas de entrada del grupo de seguridad asociado a la instancia Amazon EC2. Se permitió el acceso mediante SSH por el puerto 22 únicamente desde la IP del administrador <IP_ADMINISTRADOR>/32, con el objetivo de limitar el acceso remoto al entorno.

Además, se habilitó el tráfico HTTP por el puerto 80 para publicar el servicio web. También se abrieron, de forma restringida a la IP del administrador, los puertos 9090, 3001 y 8080. Estos puertos correspondieron a Prometheus, Uptime Kuma y cAdvisor.

Esta configuración permitió administrar y validar el entorno desplegado con una política básica de seguridad. Las reglas de entrada limitaron el tráfico autorizado hacia la instancia, de acuerdo con el funcionamiento de los grupos de seguridad de Amazon EC2 (Amazon Web Services, s. f.-c).

ID de la regla del grupo de seguridad	Tipo	Protocolo	Intervalo de puertos	Origen	Descripción opcional	Acción
sgr-04b504ab5e0a2d702	SSH	TCP	22	Persona... 37.132.183.92/32		Eliminar
sgr-0f5d6063ed9430e8	HTTP	TCP	80	Persona... 37.132.183.92/32		Eliminar
sgr-0c3660a0bc2f4d379	HTTPS	TCP	443	Persona... 0.0.0.0		Eliminar
-	TCP personalizado	TCP	9090	Persona... 37.132.183.92/32		Eliminar
-	TCP personalizado	TCP	3001	Persona... 37.132.183.92/32		Eliminar
-	TCP personalizado	TCP	8080	Persona... 37.132.183.92/32		Eliminar

⚠️ Las reglas cuyo origen es 0.0.0.0/0 o :::/0 permiten a todas las direcciones IP acceder a la instancia. Recomendamos configurar reglas de grupo de seguridad para permitir el acceso únicamente desde direcciones IP conocidas.

[Agregar regla](#) [Cancelar](#) [Previsualizar los cambios](#) [Guardar reglas](#)

Figura 6.7 : Reglas de entrada finales del grupo de seguridad de la instancia EC2.

Fuente: elaboración propia.

Instancia creada correctamente:

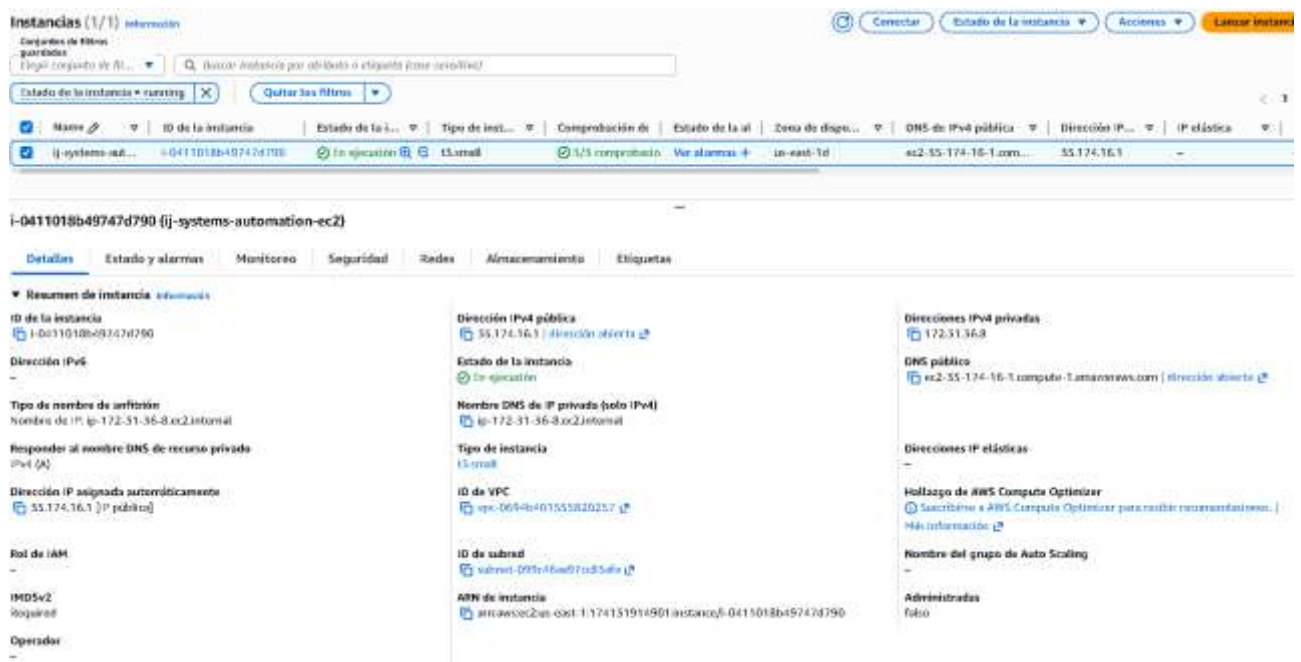


Figura 6.8 : Instancia EC2 creada y en ejecución.

Fuente: elaboración propia.

El acceso administrativo a la instancia se realizó mediante SSH:



Figura 6.9 : Instrucciones de conexión SSH a la instancia EC2.

Fuente: elaboración propia.

El archivo .pem generado durante la creación del par de claves se utilizó como credencial de acceso SSH a la instancia EC2. El comando de conexión empleado fue el siguiente (Amazon Web Services, s. f.-f):

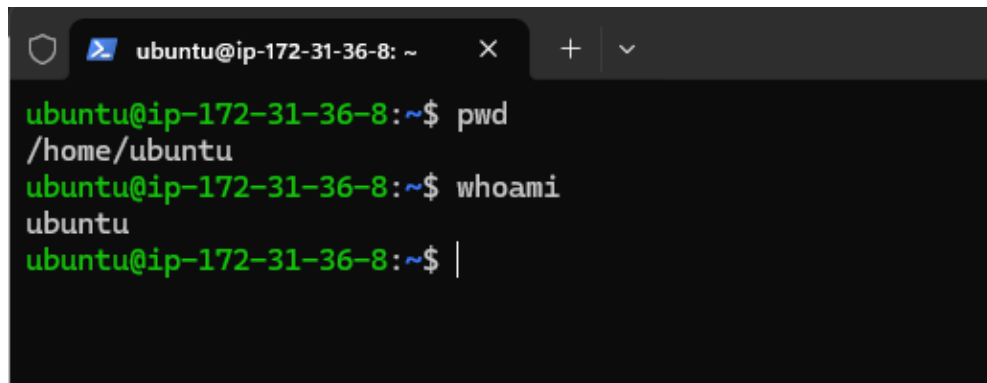
```
PS C:\Users\iangu> ssh -i "C:\Users\iangu\Downloads\IJ-claves.pem" ubuntu@
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1007-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Thu Apr 23 21:55:57 UTC 2026
```

Figura 6.10 : Conexión SSH correcta a la instancia EC2.

Fuente: elaboración propia.



```
ubuntu@ip-172-31-36-8: ~
ubuntu@ip-172-31-36-8:~$ pwd
/home/ubuntu
ubuntu@ip-172-31-36-8:~$ whoami
ubuntu
ubuntu@ip-172-31-36-8:~$ |
```

Figura 6.11 : Verificación del usuario y directorio inicial en la instancia EC2.

Fuente: elaboración propia.

Se creó una **IP elástica** para mantener una dirección pública estable durante las pruebas y la documentación del proyecto:

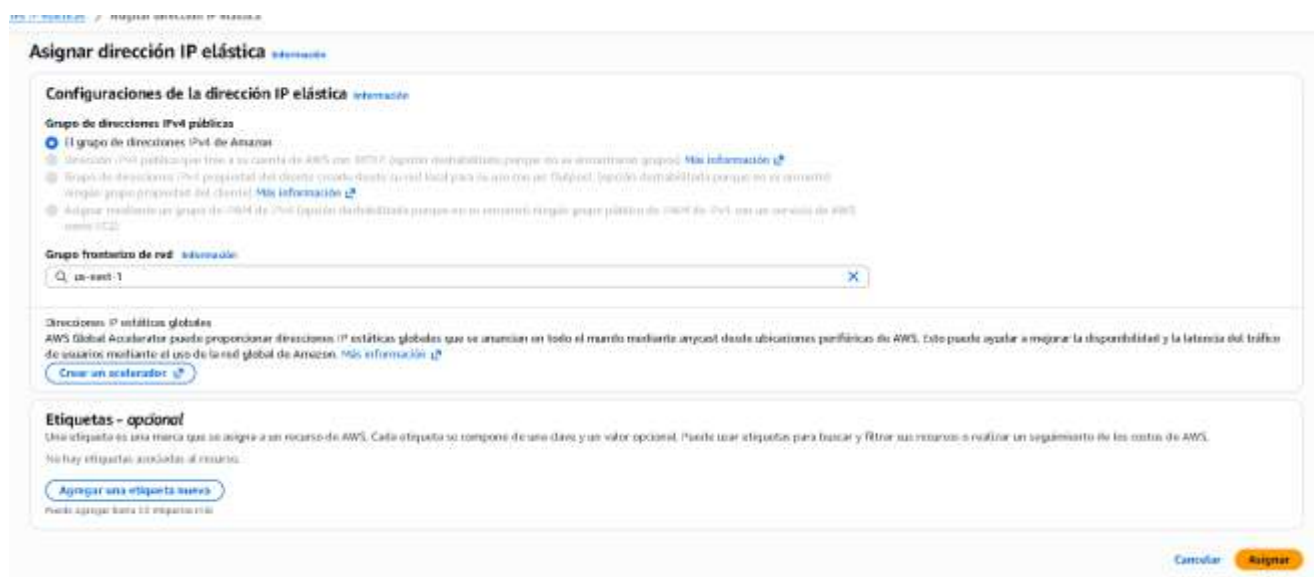


Figura 6.12 : Asignación de una IP elástica en Amazon EC2.

Fuente: elaboración propia.

Se asoció la IP elástica a la instancia EC2:



Figura 6.13 : Asociación de la IP elástica a la instancia EC2.

Fuente: elaboración propia.



Figura 6.14 : Confirmación de la asociación de la IP elástica a la instancia EC2.

Fuente: elaboración propia.

6.1 Entorno de implementación

El entorno principal de implementación estuvo compuesto por una instancia **Amazon EC2** con **Ubuntu Server LTS**. La instancia se configuró con acceso mediante clave SSH y con permisos limitados al usuario administrador. Sobre este sistema se instalaron **Docker Engine**, el plugin **Docker Compose**, **Ansible** y las utilidades necesarias para gestionar el entorno. La instalación se apoyó en la documentación oficial de Amazon Web Services, Docker y Ansible (Amazon Web Services, s. f.-g; Ansible Community, s. f.-a; Docker Inc., s. f.-c).

El proyecto se almacenó en un repositorio de **GitHub** para registrar cambios, ramas de trabajo y versiones funcionales del despliegue. Desde el punto de vista operativo, se trabajó con una estructura de directorios estable. Esta organización permitió repetir la instalación y documentar cada modificación relevante (GitHub, s. f.).

Inventario técnico del entorno:

El siguiente inventario resume los recursos y servicios que formaron parte del despliegue final, incluyendo su uso dentro del proyecto, la versión o imagen utilizada y el comando empleado para verificarla. Su finalidad fue ofrecer una visión cerrada del entorno técnico utilizado, facilitando la revisión y la repetición del despliegue.

Elemento	Uso dentro del proyecto	Versión / imagen	Comprobación
Ubuntu Server LTS	Sistema operativo base de la instancia EC2.	Ubuntu 24.04.4 LTS (noble)	lsb_release -a
Instancia Amazon EC2	Servidor principal donde se despliega el entorno automatizado.	t3.micro · región us-east-1	Consola AWS EC2
Almacenamiento de la instancia	Soporte para sistema, repositorio, ficheros de configuración y volúmenes Docker.	EBS gp3, 20 GB	df -h
Docker Engine	Motor de contenedores utilizado para ejecutar los servicios.	Docker version 27.3.1, build ce12230	docker --version
Docker Compose	Definición y arranque coordinado de la pila de contenedores.	Docker Compose version v2.29.7	docker compose version
Ansible	Automatización declarativa de la preparación del host.	ansible [core 2.16.3]	ansible --version
Python (Ansible)	Intérprete utilizado por Ansible para ejecutar módulos.	Python 3.12.3	ansible --version
Bash	Scripts de instalación, comprobación, backup y restauración.	GNU bash 5.2.21	bash --version
Nginx	Servicio web de validación publicado por HTTP.	Nginx 1.27	docker images
MariaDB	Servicio de base de datos con volumen persistente y pruebas de backup/restore.	MariaDB 11.4	docker images
Prometheus	Recogida de métricas del host y servicios configurados.	Prometheus v3.11.2	docker images
node-exporter	Exportador de métricas del host utilizado por Prometheus para consultar CPU, memoria, disco y red.	node-exporter v1.11.1	docker images
Uptime Kuma	Supervisión de disponibilidad desde interfaz web.	uptime-kuma 1.23.17	docker images
cAdvisor	Visualización de consumo de recursos de contenedores.	cAdvisor v0.55.1	docker images
GitHub	Control de versiones y trazabilidad del código fuente.	Servicio SaaS	git remote -v
Red frontend	Exposición controlada de servicios web y herramientas de validación.	Bridge Docker	docker network ls
Red interna / monitoring	Comunicación entre servicios no expuestos directamente.	Bridge Docker	docker network inspect monitoring
Volúmenes Docker	Persistencia de datos y soporte para recuperación.	Named volumes	docker volume ls

Tabla 6.1 : Inventario técnico del entorno (elementos, uso, versiones y comprobación).

Fuente: elaboración propia.

6.2 Estructuras técnicas

Dado que el proyecto no contempló el diseño de una base de datos relacional propia, este apartado describe las estructuras técnicas definidas durante la implementación.

Estructura técnica	Finalidad	Ejemplo previsto
Scripts Bash	Automatización de tareas operativas del host	instalar.sh, checks.sh, backup.sh y restore.sh
Redes Docker	Separación lógica del tráfico	frontend y monitoring
Volúmenes	Persistencia y copias	Datos de MariaDB
Inventario Ansible	Identificación del host	Grupo con instancia EC2 objetivo

Tabla 6.2 : Estructuras técnicas.

Fuente: elaboración propia.

6.3 Carga de datos

La carga de datos no consistió en introducir información de negocio real, sino en preparar la información técnica mínima necesaria para que el entorno arrancara, pudiera comprobarse y quedara validado funcionalmente. En esta fase se prepararon los siguientes elementos:

- Fichero `.env`, con las variables básicas de despliegue, puertos de acceso y credenciales iniciales del servicio MariaDB. Para la entrega final, este fichero se trató como configuración local del entorno y no como información destinada a publicarse en el repositorio.
- Para reforzar la seguridad, las credenciales reales no se publicaron como parte del repositorio final. En un uso mantenido del proyecto, el fichero `.env` debe quedar excluido mediante `.gitignore` y sustituirse por un fichero `.env.example` sin contraseñas reales, de forma que se mantenga la reproducibilidad sin exponer secretos.
- Contenido estático de prueba para **Nginx**, utilizado para validar que la publicación web funcionaba correctamente desde el navegador.
- Fichero `prometheus.yml`, con la definición de los trabajos de recogida de métricas para **Prometheus**, **node-exporter** y **cAdvisor** (Prometheus Authors, s. f.-d; Google, s. f.).
- Inventario y variables de **Ansible**, con la información necesaria para la automatización del host objetivo (Ansible Community, s. f.-c).
- Configuración inicial de supervisión en **Uptime Kuma**, orientada a comprobar la disponibilidad del servicio web publicado (Lam, s. f.).
- Datos de prueba en **MariaDB**, mediante la creación de la base de datos `ijdb`, una tabla de validación llamada `prueba` y la inserción de un registro de ejemplo (`dato1`), con el objetivo de demostrar conectividad, escritura, lectura y persistencia tras el reinicio del contenedor. La validación se planteó de acuerdo con el uso del cliente de línea de

comandos y las herramientas de copia de seguridad documentadas por MariaDB (MariaDB plc, s. f.-a).⁷

No se manejó información real ni sensible. La finalidad de esta carga inicial fue demostrar que el entorno desplegado permitía validar correctamente la conectividad, la automatización, la persistencia de datos y la recuperación básica del servicio.

Además, todas las capturas incluidas en la memoria han sido revisadas para que las contraseñas, tokens, direcciones personales o datos sensibles aparezcan ocultos o sustituidos por marcadores genéricos. Esta medida permite conservar evidencias reales del funcionamiento del sistema sin comprometer la seguridad del entorno ni del repositorio.

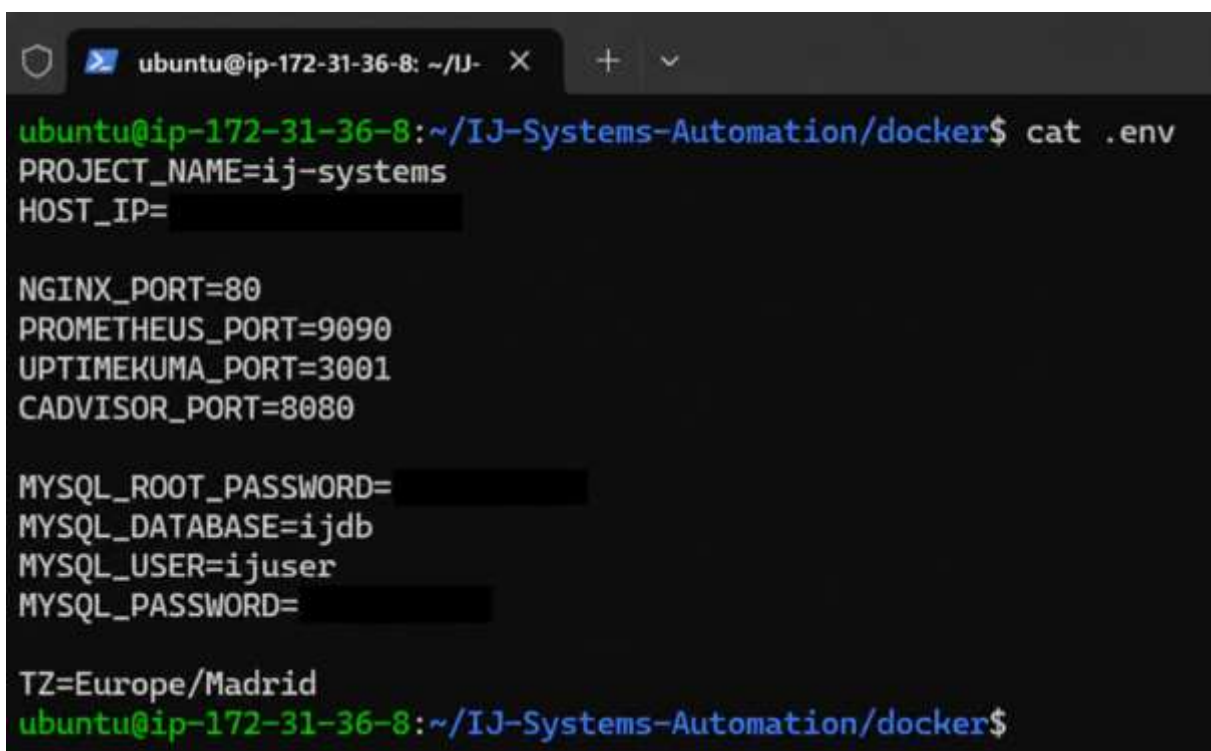
A screenshot of a terminal window with a dark background. The terminal shows the command 'cat .env' being executed in a directory '~/.IJ-Systems-Automation/docker'. The output lists several environment variables: PROJECT_NAME=ij-systems, HOST_IP= (redacted), NGINX_PORT=80, PROMETHEUS_PORT=9090, UPTIMEKUMA_PORT=3001, CADVISOR_PORT=8080, MYSQL_ROOT_PASSWORD= (redacted), MYSQL_DATABASE=ijdb, MYSQL_USER=ijuser, MYSQL_PASSWORD= (redacted), and TZ=Europe/Madrid. The prompt at the bottom is 'ubuntu@ip-172-31-36-8: ~/.IJ-Systems-Automation/docker\$'.

Figura 6.15 : Variables de entorno definidas en el fichero .env.

Fuente: elaboración propia.

Como medida adicional de seguridad, se revisó la gestión de credenciales del proyecto. El objetivo fue evitar que contraseñas reales quedaran publicadas en el repositorio. Para ello, el fichero docker/.env se mantuvo únicamente como configuración local del entorno y se excluyó del control de versiones mediante el fichero .gitignore.

Además, se creó el fichero docker/.env.example como plantilla segura. Las contraseñas reales se sustituyeron por marcadores genéricos. De esta forma, el proyecto puede reproducirse en otro entorno sin exponer credenciales sensibles. También se modificaron los ficheros docker-compose.yml, backup.sh y restore.sh para utilizar variables de entorno en lugar de contraseñas escritas directamente en el código.

⁷ Documentación oficial del cliente de línea de comandos de MariaDB. Disponible en: <https://mariadb.com/docs/server/clients-and-utilities/mariadb-client/mariadb-command-line-client>. Referencia para las operaciones de validación, copia y restauración.

Finalmente, se realizó una comprobación mediante comandos de Git para verificar que el fichero `.env` aparecía como ignorado y que no quedaban credenciales reales en los archivos versionados.

```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation$ cat .gitignore
# Copias de seguridad y archivos generados
backups/*.sql
*.log
*.tmp
*.swp
.DS_Store

# Variables locales y credenciales
.env
**/.env
*.env
docker/.env

# Se permite subir solo plantillas sin contraseñas reales
!.env.example
!docker/.env.example
!*.env.example
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ cat docker/.env.example
PROJECT_NAME=ij-systems
HOST_IP=<IP_DEL_SERVIDOR>

NGINX_PORT=80
PROMETHEUS_PORT=9090
UPTIMEKUMA_PORT=3001
CADVISOR_PORT=8080

MYSQL_ROOT_PASSWORD=<CAMBIAR_CONTRASENA_ROOT>
MYSQL_DATABASE=ijdb
MYSQL_USER=ijuser
MYSQL_PASSWORD=<CAMBIAR_CONTRASENA_USUARIO>

TZ=Europe/Madrid
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ git status --short --ignored docker/.env docker/.env.example .gitignore
!! docker/.env
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ git grep -n "\|" " || true
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ git grep -n "MYSQL_ROOT_PASSWORD\|MYSQL_PASSWORD"
docker/.env.example:9:MYSQL_ROOT_PASSWORD=<CAMBIAR_CONTRASENA_ROOT>
docker/.env.example:12:MYSQL_PASSWORD=<CAMBIAR_CONTRASENA_USUARIO>
docker/docker-compose.yml:38:    MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
docker/docker-compose.yml:33:    MYSQL_PASSWORD: ${MYSQL_PASSWORD}
scripts/backup.sh:11:docker exec mariadb mariadb-dump -u root -p"${MYSQL_ROOT_PASSWORD}" ijdb > "$ARCHIVO"
scripts/restore.sh:19:docker exec -i mariadb mariadb -u root -p"${MYSQL_ROOT_PASSWORD}" ijdb < "$ARCHIVO"
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ |
```

Figura 6.16 : Comprobación de exclusión del fichero `.env` y uso de variables de entorno.

Fuente: elaboración propia.

```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation$ cat monitoring/prometheus/prometheus.yml
global:
  scrape_interval: 15s

scrape_configs:
  - job_name: 'prometheus'
    static_configs:
      - targets: ['172.30.30.10:9090']

  - job_name: 'node-exporter'
    static_configs:
      - targets: ['172.30.30.30:9100']

  - job_name: 'cadvisor'
    static_configs:
      - targets: ['172.30.30.40:8080']
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation$
```

Figura 6.17 : Configuración de Prometheus para la recogida de métricas.

Fuente: elaboración propia.

```
ubuntu@ip-172-31-36-8: ~/IJ- X + v
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/docker/nginx$ cat index.html
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>IJ Systems Automation</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }

    body {
      font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif;
      background: #0f1117;
      color: #e2e8f0;
      min-height: 100vh;
      display: flex;
      flex-direction: column;
      align-items: center;
      padding: 40px 20px;
    }

    /* HEADER */
    header {
      text-align: center;
      margin-bottom: 50px;
    }

    .logo {
      font-size: 3rem;
      margin-bottom: 10px;
    }

    header h1 {
      font-size: 2.4rem;
      font-weight: 700;
      background: linear-gradient(90deg, #38bdf8, #818cf8, #f472b6);
      -webkit-background-clip: text;
      -webkit-text-fill-color: transparent;
      background-clip: text;
    }

    header p {
      color: #94a3b8;
      margin-top: 8px;
      font-size: 1rem;
      letter-spacing: 0.05em;
    }
  </style>
</html>
```

Figura 6.18 : Parte del contenido HTML de prueba publicado mediante Nginx.

Fuente: elaboración propia.

```

ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/docker$ docker exec -it mariadb mariadb -u root -p
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 5
Server version: 11.4.5-MariaDB mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help, Type '\c' to clear the current input statement.

MariaDB [(none)]> SHOW DATABASES;
+-----+
| Database |
+-----+
| ijdb     |
| information_schema |
| mysql    |
| performance_schema |
| sys      |
+-----+
5 rows in set (0.001 sec)

MariaDB [(none)]> USE ijdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [ijdb]> CREATE TABLE prueba (
  -> id INT AUTO_INCREMENT PRIMARY KEY,
  -> nombre VARCHAR(50)
  -> );INSERT INTO prueba (nombre) VALUES ('dato1');
Query OK, 1 row affected (0.001 sec)

MariaDB [ijdb]> SELECT * FROM prueba;
+----+-----+
| id | nombre |
+----+-----+
| 1  | dato1  |
| 2  | dato1  |
| 3  | dato1  |
+----+-----+
3 rows in set (0.001 sec)

MariaDB [ijdb]> EXIT;
Bye
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/docker$

```

Figura 6.19 : Validación de la base de datos MariaDB desde consola.

Fuente: elaboración propia.

6.4 Ficheros de configuración actualizados

Durante la implementación se generaron y actualizaron los principales ficheros de configuración del proyecto. La estructura se organizó para que resultara clara, versionable y fácil de revisar por bloques.

Fichero	Contenido principal	Finalidad
docker-compose.yml	Servicios, redes, puertos y volúmenes	Levantar la pila completa
.env local no versionado	Variables de entorno reales del entorno local	Centralizar parámetros sensibles sin publicarlos en el repositorio
.env.example	Plantilla de variables sin contraseñas reales	Facilitar la reproducción del proyecto de forma segura
prometheus.yml	Targets y jobs	Recoger métricas del host y contenedores
inventory.ini	Host objetivo de Ansible	Automatizar la configuración
playbook.yml	Tareas declarativas	Instalar y dejar preparado el servidor
docker/nginx/nginx.conf	Configuración del proxy inverso de Nginx	Enrutar el tráfico hacia los servicios de monitorización internos

Tabla 6.3 : Ficheros de configuración.

Fuente: elaboración propia.

El fichero .env se utiliza únicamente como configuración local del entorno y no debe incluirse en el repositorio. Para mantener la reproducibilidad del proyecto sin exponer credenciales, se proporciona un fichero .env.example con la misma estructura y con valores genéricos.

6.5 Configuraciones realizadas en el sistema

En el sistema se realizaron las siguientes configuraciones principales: actualización del host, instalación de **Docker Engine** y del plugin **Docker Compose**, alta del usuario en el grupo docker, preparación de directorios persistentes y despliegue de los contenedores. También se verificó la conectividad entre servicios (Docker Inc., s. f.-c).

- Apertura controlada de puertos para **SSH** y para los servicios web necesarios.
- Separación de redes Docker por función, con el objetivo de ordenar el tráfico interno del entorno.
- Creación de rutas de copia de seguridad y preparación de scripts para la generación de backups.
- Comprobaciones de estado mediante docker ps, validaciones de contenedores y supervisión externa.
- Preparación de servicios para representar métricas de CPU, memoria, red y disponibilidad.

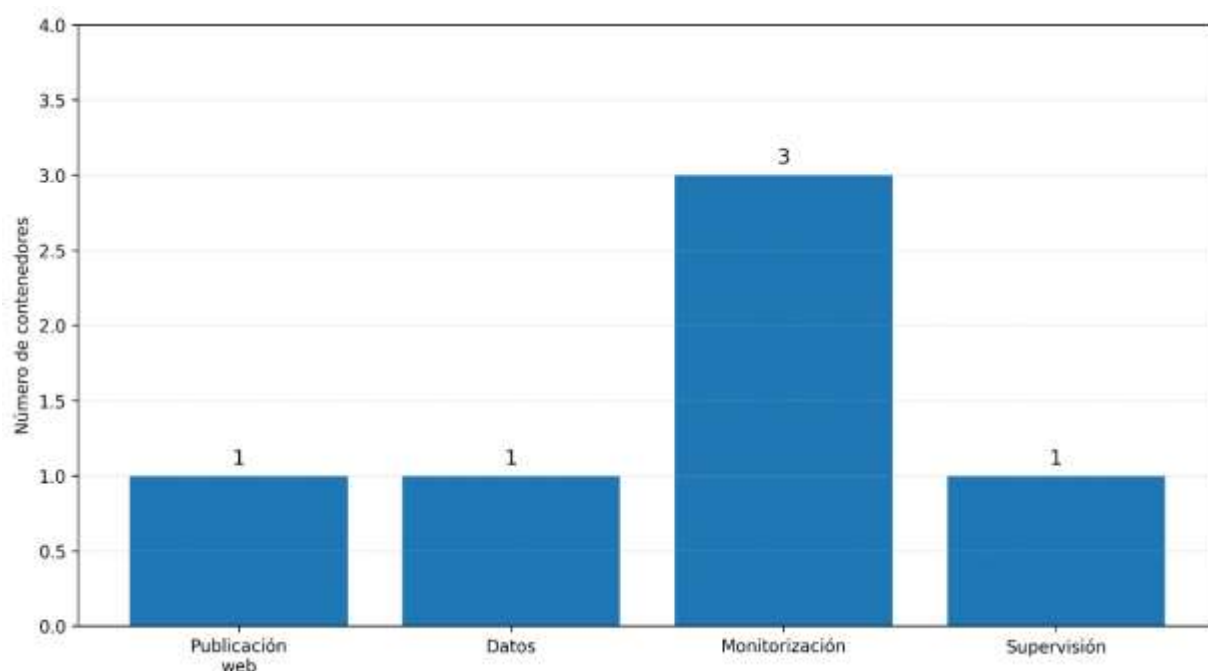


Figura 6.20 : Distribución funcional de los contenedores del proyecto.

Fuente: elaboración propia.

La figura anterior no representa métricas reales de producción, sino la distribución funcional de la pila desplegada. Se incluyó porque ayuda a visualizar el peso relativo de cada bloque del despliegue y facilita la comprensión global de la arquitectura implementada.

6.6 Implementaciones de código realizadas

El código desarrollado se organizó en tres bloques principales. Los scripts Bash cubrieron automatizaciones locales y tareas rápidas. Los ficheros de Ansible permitieron aplicar tareas declarativas. Docker Compose concentró el arranque coordinado de los servicios. Esta separación facilitó las pruebas, el mantenimiento y la reutilización del proyecto.

Bloque de código	Contenido previsto	Objetivo
Scripts Bash	instalar.sh, backup.sh, checks.sh, restore.sh	Automatizar tareas repetitivas del host
Ansible	inventory.ini y playbook.yml	Configurar el servidor de forma reproducible
Docker Compose	docker-compose.yml	Levantar y detener la pila de contenedores
Monitorización	prometheus.yml, configuración de Uptime Kuma y acceso a cAdvisor	Supervisar métricas, disponibilidad de servicios y estado de los contenedores

Tabla 6.4 : Implementaciones de código.

Fuente: elaboración propia.

La monitorización del entorno se apoyó en **Prometheus**, **cAdvisor** y **Uptime Kuma**. Prometheus recogió métricas técnicas. cAdvisor permitió visualizar el uso de recursos de los contenedores. Uptime Kuma se utilizó para supervisar la disponibilidad externa de los

servicios. Esta combinación permitió comprobar el estado general del despliegue sin añadir plataformas de paneles adicionales (Prometheus Authors, s. f.-d; Google, s. f.; Lam, s. f.).^{8 9}

10

Desde el punto de vista de la seguridad, los puertos expuestos se limitaron a los servicios necesarios para validar el proyecto. El acceso administrativo y las herramientas de monitorización se restringieron a la IP del administrador siempre que fue posible. Con ello se siguió el principio de mínima exposición y el modelo de control de tráfico mediante grupos de seguridad de Amazon EC2 (Amazon Web Services, s. f.-c).

Servicio	Puerto	Exposición	Justificación
SSH	22	Solo IP del administrador	Administración remota de la instancia mediante clave privada
Nginx	80	Público	Publicación de la página web de validación del proyecto
HTTPS	443	No utilizado en la versión actual	Queda contemplado como ampliación futura para una publicación HTTPS con certificado TLS, pero no forma parte de la validación funcional principal de esta versión.
Prometheus	9090	Solo IP del administrador	Consulta de métricas internas del entorno
node-exporter	9100	Solo red interna / monitoring	Exportación de métricas del host para consulta desde Prometheus, sin exposición pública directa
Uptime Kuma	3001	Solo IP del administrador	Supervisión de disponibilidad de los servicios
cAdvisor	8080	Solo IP del administrador	Visualización de métricas de contenedores
MariaDB	3306	No expuesto públicamente	Acceso únicamente desde el contenedor o red interna
Nginx (proxy inverso)	80	Público	Enruta el tráfico hacia Prometheus (/prometheus/) y cAdvisor (/metrics/) sin exponer sus puertos directamente

Tabla 6.5 : Puertos, exposición y justificación de seguridad.

Fuente: elaboración propia.

⁸ Documentación oficial de Prometheus, sistema de monitorización utilizado en el proyecto. Disponible en: <https://prometheus.io/docs/introduction/overview/>.

⁹ Repositorio oficial de cAdvisor desarrollado por Google. Disponible en: <https://github.com/google/cadvisor>. Herramienta utilizada para visualizar las métricas de los contenedores Docker desplegados.

¹⁰ Repositorio oficial de Uptime Kuma, mantenido por Louis Lam. Disponible en: <https://github.com/louislam/uptime-kuma>. Herramienta empleada para supervisar la disponibilidad de los servicios web.

Adicionalmente, los servicios de monitorización son accesibles a través del proxy inverso de Nginx por el puerto 80, sin necesidad de exponer sus puertos internos. De esta forma, Prometheus queda disponible en `http://<IP>/prometheus/` y cAdvisor en `http://<IP>/metrics/`. Uptime Kuma (v1.23.17) no admite de forma nativa el servicio bajo un sub-path, por lo que se mantiene accesible directamente en el puerto :3001, documentado como decisión técnica justificada.

Scripts Bash implementados:

Se prepararon cuatro scripts orientados a la instalación, la comprobación del entorno, la copia de seguridad y la restauración. La lógica aplicada permitió reutilizar el proyecto sin rehacer tareas manuales repetitivas y facilitó la explotación básica del entorno desplegado.



Figura 6.21 : Resumen visual de los scripts Bash del proyecto.

Fuente: elaboración propia.

```

ubuntu@ip-172-31-36-8: ~ X + v
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$ cat backup.sh
#!/bin/bash
set -a
source "$(dirname "$0")/../../docker/.env"
set +a

BACKUP_DIR="$HOME/IJ-Systems-Automation/backups"
FECHA=$(date +%F_%H-%M-%S)
ARCHIVO="$BACKUP_DIR/backup_-$FECHA.sql"
DIAS_RETENCION=7

mkdir -p "$BACKUP_DIR"

echo "Realizando backup de ijdbc..."
docker exec mariadb mariadb-dump -u root -p"$MYSQL_ROOT_PASSWORD" ijdbc > "$ARCHIVO"

if [ $? -eq 0 ]; then
    TAMANO=$(du -sh "$ARCHIVO" | cut -f1)
    echo "Backup completado: $ARCHIVO ($TAMANO)"
else
    echo "Error al realizar el backup"
    rm -f "$ARCHIVO"
    exit 1
fi

echo "Eliminando backups con mas de $DIAS_RETENCION dias..."
find "$BACKUP_DIR" -name "backup_*.sql" -mtime +$DIAS_RETENCION -delete
RESTANTES=$(ls "$BACKUP_DIR"/backup_*.sql 2>/dev/null | wc -l)
echo "Backups disponibles: $RESTANTES"
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$ |

```

Figura 6.22 : Script Bash para la copia de seguridad de MariaDB.

Fuente: elaboración propia.

```

ubuntu@ip-172-31-36-8: ~/IJ X + v
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$ cat instalar.sh
#!/bin/bash

echo "Actualizando sistema..."
sudo apt update && sudo apt upgrade -y

echo "Instalando herramientas base..."
sudo apt install -y git curl wget unzip net-tools htop tree ansible

if command -v docker &> /dev/null; then
    echo "Docker ya esta instalado: $(docker --version)"
else
    echo "Instalando Docker desde repositorio oficial..."
    sudo apt install -y ca-certificates curl gnupg
    sudo install -m 0755 -d /etc/apt/keyrings
    curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
    sudo chmod a+r /etc/apt/keyrings/docker.gpg

    echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo "$VERSION_CODENAME") stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null

    sudo apt update
    sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin

    echo "Añadiendo usuario al grupo docker..."
    sudo usermod -aG docker $USER
    echo "Cierra sesion y vuelve a entrar para usar Docker sin sudo."
fi

echo ""
echo "Instalacion finalizada."
echo "Versions instaladas:"
docker --version
docker compose version
ansible --version | head -n 1
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$ |

```

Figura 6.23 : Script Bash de instalación y preparación del servidor.

Fuente: elaboración propia.

```

ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/scripts$ cat checks.sh
#!/bin/bash
set -a
source "${dirname "$0"}/../docker/.env"
set +a

echo "===== VERSIONES ====="
docker --version
docker compose version
ansible --version | head -n 1

echo
echo "===== CONTENEDORES ====="
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"

echo
echo "===== REDES ====="
docker network ls

echo
echo "===== VOLUMENES ====="
docker volume ls

echo
echo "===== PRUEBAS HTTP ====="

echo "Nginx (puerto 80):"
curl -s -o /dev/null -w " HTTP %{http_code} en %{time_total}s\n" http://localhost || echo " No responde"

echo "Prometheus (puerto 9090):"
curl -s -o /dev/null -w " HTTP %{http_code} en %{time_total}s\n" http://localhost:9090 || echo " No responde"

echo "Uptime Muma (puerto 3001):"
curl -s -o /dev/null -w " HTTP %{http_code} en %{time_total}s\n" http://localhost:3001 || echo " No responde"

echo "cAdvisor (puerto 8080):"
curl -s -o /dev/null -w " HTTP %{http_code} en %{time_total}s\n" http://localhost:8080 || echo " No responde"

echo
echo "===== BASE DE DATOS ====="
docker exec mariadb mariadb -u root -p"${MYSQL_ROOT_PASSWORD}" -e "SELECT 'MariaDB operativa' AS estado;" 2>/dev/null && echo " Conexión correcta" || echo " No se pudo conectar a MariaDB"

echo
echo "===== USO DE RECURSOS ====="
docker stats --no-stream --format "table {{.Name}}\t{{.CPUPerc}}\t{{.MemUsage}}"

echo
echo "===== FIN DE COMPROBACIONES ====="
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$

```

Figura 6.24 : Script Bash de comprobación del entorno desplegado.
Fuente: elaboración propia.

```

ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/scripts$ cat restore.sh
#!/bin/bash
set -a
source "${dirname "$0"}/../docker/.env"
set +a

BACKUP_DIR="$HOME/IJ-Systems-Automation/backups"

if [ -z "$1" ]; then
    echo "Uso: ./restore.sh nombre_del_backup.sql"
    echo ""
    echo "Backups disponibles:"
    ls -lh "$BACKUP_DIR"/backup_*.sql 2>/dev/null | awk '{print " " $NF " (" $5 ")"}' || echo " No hay backups disponibles"
    exit 1
fi

ARCHIVO="$BACKUP_DIR/$1"

if [ ! -f "$ARCHIVO" ]; then
    echo "Error: no existe el archivo: $ARCHIVO"
    echo ""
    echo "Backups disponibles:"
    ls "$BACKUP_DIR"/backup_*.sql 2>/dev/null || echo " No hay backups disponibles"
    exit 1
fi

echo "Restaurando $1 en ijdbc..."
docker exec -i mariadb mariadb -u root -p"${MYSQL_ROOT_PASSWORD}" ijdbc < "$ARCHIVO"

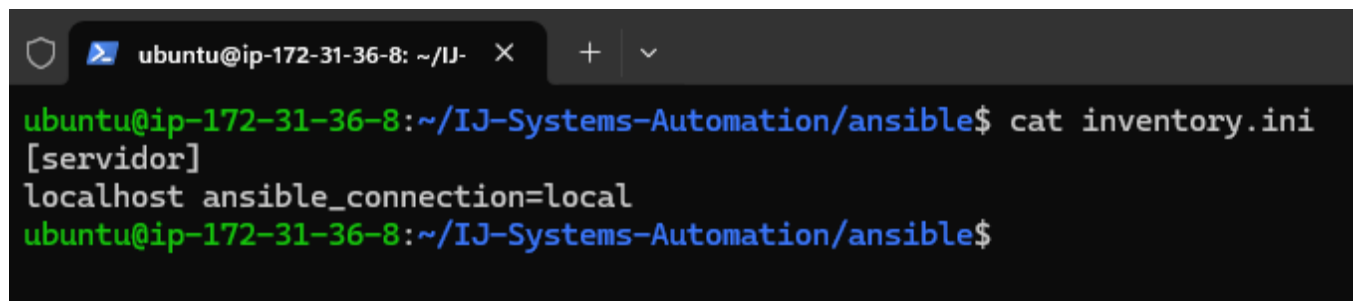
if [ $? -eq 0 ]; then
    echo "Restauración completada correctamente"
else
    echo "Error durante la restauración"
    exit 1
fi
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$

```

Figura 6.25 : Script Bash de restauración de copias de seguridad de MariaDB.
Fuente: elaboración propia.

Automatización con Ansible:

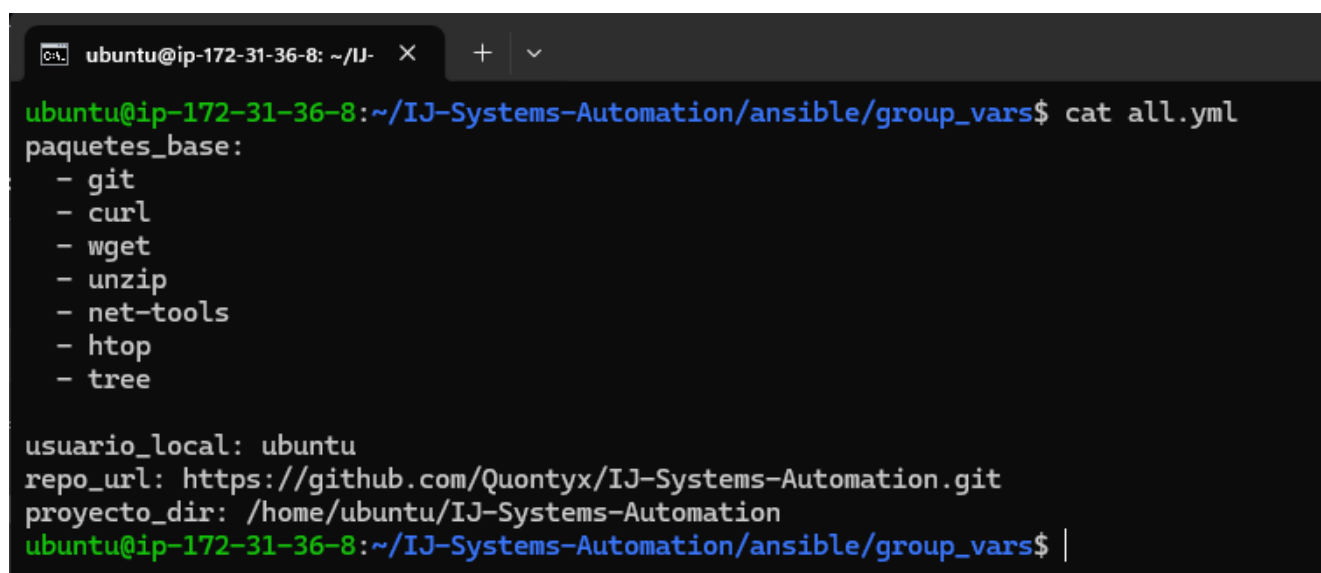
La automatización declarativa se apoyó en un inventario local de pruebas, un fichero global de variables y un playbook orientado a la actualización de paquetes, la preparación del host y el alta del usuario en el grupo docker (Ansible Community, s. f.-b).

A terminal window with a dark background. The prompt is 'ubuntu@ip-172-31-36-8: ~/IJ-'. The user has entered 'cat inventory.ini'. The output shows '[servidor]' followed by 'localhost ansible_connection=local' on the next line.

```
ubuntu@ip-172-31-36-8: ~/IJ-  
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/ansible$ cat inventory.ini  
[servidor]  
localhost ansible_connection=local  
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/ansible$
```

Figura 6.26 : Inventario de Ansible utilizado para la configuración local.

Fuente: elaboración propia.

A terminal window with a dark background. The prompt is 'ubuntu@ip-172-31-36-8: ~/IJ-'. The user has entered 'cat all.yml'. The output shows a list of packages under 'paquetes_base:', followed by 'usuario_local: ubuntu', 'repo_url: https://github.com/Quontyx/IJ-Systems-Automation.git', and 'proyecto_dir: /home/ubuntu/IJ-Systems-Automation'.

```
ubuntu@ip-172-31-36-8: ~/IJ-  
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/ansible/group_vars$ cat all.yml  
paquetes_base:  
- git  
- curl  
- wget  
- unzip  
- net-tools  
- htop  
- tree  
  
usuario_local: ubuntu  
repo_url: https://github.com/Quontyx/IJ-Systems-Automation.git  
proyecto_dir: /home/ubuntu/IJ-Systems-Automation  
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/ansible/group_vars$ |
```

Figura 6.27 : Variables globales utilizadas por Ansible.

Fuente: elaboración propia.

El playbook de Ansible automatiza de forma declarativa la preparación y el despliegue del servidor objetivo, definido en el inventario bajo el grupo servidor y ejecutado con privilegios elevados (become: true).

A partir del fichero de variables group_vars/all.yml, las tareas se encadenan en un orden lógico que reproduce la puesta en marcha del entorno sin intervención manual: primero actualiza la caché de paquetes APT e instala las utilidades base definidas en la variable paquetes_base, después se asegura de que el servicio Docker esté iniciado y habilitado en el arranque, y añade el usuario local al grupo docker para permitir la gestión de contenedores sin privilegios de superusuario.

A continuación, clona el repositorio del proyecto desde la rama main en el directorio de trabajo, copia el fichero de variables de entorno con permisos restringidos (0600) para

proteger las credenciales, y levanta la pila completa de servicios mediante el módulo `community.docker.docker_compose_v2`.

Por último, configura una tarea programada con `cron` que ejecuta diariamente el script `backup.sh` y registra su salida en un fichero de log, garantizando que las copias de seguridad de MariaDB se generen de forma desatendida. En la siguiente Figura 6.28 se puede ver el archivo `playbook.yml`.

```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/ansible$ cat playbook.yml
- name: Preparacion y despliegue del servidor local
  hosts: servidor
  become: true

  vars_files:
    - group_vars/all.yml

  tasks:
    - name: Actualizar cache APT
      apt:
        update_cache: yes

    - name: Instalar paquetes base
      apt:
        name: "{{ paquetes_base }}"
        state: present

    - name: Asegurar que Docker esta iniciado
      service:
        name: docker
        state: started
        enabled: true

    - name: Añadir usuario local al grupo docker
      user:
        name: "{{ usuario_local }}"
        groups: docker
        append: yes

    - name: Clonar repositorio del proyecto
      git:
        repo: "{{ repo_url }}"
        dest: "{{ proyecto_dir }}"
        version: main
        force: yes
        become_user: "{{ usuario_local }}"

    - name: Copiar fichero de variables de entorno
      copy:
        src: files/.env
        dest: "{{ proyecto_dir }}/docker/.env"
        owner: "{{ usuario_local }}"
        mode: '0600'

    - name: Levantar servicios con Docker Compose
      community.docker.docker_compose_v2:
        project_src: "{{ proyecto_dir }}/docker"
        state: present
        become_user: "{{ usuario_local }}"

    - name: Configurar cron de backup automatico
      cron:
        name: "Backup diario MariaDB"
        minute: "0"
        hour: "2"
        job: "{{ proyecto_dir }}/scripts/backup.sh >> {{ proyecto_dir }}/backups/backup.log 2>&1"
        user: "{{ usuario_local }}"

ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/ansible$ |
```

Figura 6.28 : Playbook de Ansible para la preparación y despliegue del servidor.

Fuente: elaboración propia.

Orquestación y monitorización:

El despliegue se concentró en el fichero `docker-compose.yml`, donde se definieron los servicios, puertos, redes y volúmenes necesarios para la ejecución de la pila de contenedores. Además, se dejaron configurados **Prometheus** y **cAdvisor** para la recogida y visualización de métricas del entorno (Docker Inc., s. f.-a; Prometheus Authors, s. f.-a; Google, s. f.).

```

ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/docker$ cat docker-compose.yml
services:
  nginx:
    image: nginx:1.27
    container_name: nginx
    ports:
      - "80:80"
    volumes:
      - ./nginx/index.html:/usr/share/nginx/html/index.html:ro
      - ./nginx/nginx.conf:/etc/nginx/conf.d/default.conf:ro
    networks:
      frontend:
        ipv4_address: 172.30.10.10
      monitoring:
        ipv4_address: 172.30.30.5
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost"]
      interval: 30s
      timeout: 5s
      retries: 3

  uptime-kuma:
    image: louislam/uptime-kuma:1.23.17
    container_name: uptime-kuma
    ports:
      - "3001:3001"
    volumes:
      - uptime_kuma_data:/app/data
    networks:
      frontend:
        ipv4_address: 172.30.10.20
    restart: unless-stopped

  mariadb:
    image: mariadb:11.4
    container_name: mariadb
    environment:
      MYSQL_ROOT_PASSWORD: ${MYSQL_ROOT_PASSWORD}
      MYSQL_DATABASE: "ijdb"
      MYSQL_USER: "ijuser"
      MYSQL_PASSWORD: ${MYSQL_PASSWORD}
      TZ: "Europe/Madrid"
    volumes:
      - mariadb_data:/var/lib/mysql
    networks:
      backend:
        ipv4_address: 172.30.20.10
    restart: unless-stopped
    healthcheck:
      test: ["CMD", "healthcheck.sh", "--connect", "--innodb_initialized"]
      interval: 30s
      timeout: 5s
      retries: 3

  prometheus:
    image: prom/prometheus:v3.11.2
    container_name: prometheus
    ports:
      - "9090:9090"
    command:
      - --config.file=/etc/prometheus/prometheus.yml
      - --web.external-url=http://34.236.28.238/prometheus/
      - --web.route-prefix=/prometheus/
    volumes:
      - ../monitoring/prometheus/prometheus.yml:/etc/prometheus/prometheus.yml:ro
    networks:
      monitoring:
        ipv4_address: 172.30.30.10
    restart: unless-stopped

```

Figura 6.29 : Primera parte del fichero `docker-compose.yml`.

Fuente: elaboración propia.

```

node-exporter:
  image: prom/node-exporter:v1.11.1
  container_name: node-exporter
  command:
    - --path.rootfs=/host
  volumes:
    - /:/host:ro,rslave
  networks:
    monitoring:
      ipv4_address: 172.30.30.30
  restart: unless-stopped

cadvisor:
  image: gcr.io/cadvisor/cadvisor:v0.55.1
  container_name: cadvisor
  ports:
    - "8080:8080"
  command:
    - --url_base_prefix=/metrics
  volumes:
    - /:/rootfs:ro
    - /var/run:/var/run:ro
    - /sys:/sys:ro
    - /var/lib/docker:/var/lib/docker:ro
  networks:
    monitoring:
      ipv4_address: 172.30.30.40
  restart: unless-stopped

networks:
  frontend:
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.10.0/24

  backend:
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.20.0/24

  monitoring:
    driver: bridge
    ipam:
      config:
        - subnet: 172.30.30.0/24

volumes:
  mariadb_data:
  uptime_kuma_data:
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/docker$

```

Figura 6.30 : Definición final de servicios, redes y volúmenes en docker-compose.yml.

Fuente: elaboración propia.

El fichero nginx.conf configura Nginx como proxy inverso del entorno, escuchando en el puerto 80 y enrutando el tráfico hacia los servicios internos publicados mediante rutas controladas.

La ruta raíz publica la página principal de validación, mientras que las rutas /prometheus/ y /metricas/ redirigen a Prometheus y cAdvisor. Uptime Kuma se mantiene accesible directamente mediante el puerto :3001, debido a su limitación con sub-path en esta versión.

En las rutas proxy configuradas se añaden cabeceras como Host, X-Real-IP y X-Forwarded-For para conservar la información del cliente original.



```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/docker/nginx$ cat nginx.conf
server {
    listen 80;
    server_name _;

    # -----
    # Web principal de validacion del TFG
    # -----
    location / {
        root /usr/share/nginx/html;
        index index.html;
    }

    # -----
    # Prometheus - accesible mediante proxy inverso en /prometheus
    # -----
    location /prometheus {
        proxy_pass          http://172.30.30.10:9090;
        proxy_set_header    Host $host;
        proxy_set_header     X-Real-IP $remote_addr;
        proxy_set_header     X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # -----
    # cAdvisor - accesible mediante proxy inverso en /metricas
    # -----
    location /metricas {
        proxy_pass          http://172.30.30.40:8080;
        proxy_set_header    Host $host;
        proxy_set_header     X-Real-IP $remote_addr;
        proxy_set_header     X-Forwarded-For $proxy_add_x_forwarded_for;
    }

    # -----
    # Uptime Kuma - NO se expone mediante sub-path en esta version.
    # Se mantiene accesible directamente en http://<IP_SERVIDOR>:3001
    # Motivo: Uptime Kuma presenta incompatibilidades con el montaje
    # bajo sub-path en Nginx en esta version del TFG.
    # -----
}

ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/docker/nginx$ |
```

Figura 6.31 : Configuración del proxy inverso en nginx.conf.

Fuente: elaboración propia.

Ejecución real del entorno:

El despliegue no quedó en un plano teórico. Se levantó la pila completa, se verificó el estado de los contenedores y se confirmó el acceso a los servicios de monitorización y supervisión definidos en el proyecto.

```

ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/docker$ docker compose up -d
[+] up 6/6
✓Container uptime-huma   Running
✓Container mariadb      Running
✓Container prometheus    Running
✓Container node-exporter Running
✓Container cadvisor      Running
✓Container nginx         Running

ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/docker$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS                               NAMES
2fda39e3c435   nginx:latest                "/docker-entrypoint..." 5 minutes ago  Up 5 minutes  0.0.0.0:80->80/tcp, [::]:80->80/tcp  nginx
d6f368a3d623   louislan/uptime-huma:latest  "/usr/bin/dumb-init ..." 5 minutes ago  Up 5 minutes  0.0.0.0:1001->1001/tcp, [::]:1001->1001/tcp  uptime-huma
f03bd2878c6e   mariadb:latest              "/docker-entrypoint..." 5 minutes ago  Up 33 seconds  3306/tcp                             mariadb
eb87ae89dfb9   prom/prometheus:latest       "/bin/prometheus --c..." 5 minutes ago  Up 5 minutes  0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp  prometheus
4156be7bb37f   prom/node-exporter:latest    "/bin/node_exporter ..." 5 minutes ago  Up 5 minutes  9100/tcp                             node-exporter
f761aade6698   gcr.io/cadvisor/cadvisor:latest "/usr/bin/cadvisor ..." 5 minutes ago  Up 5 minutes  0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp  cadvisor
  
```

Figura 6.32 : Despliegue correcto de la pila de contenedores con Docker Compose.

Fuente: elaboración propia.

En la **Figura 6.32** se muestra la ejecución del comando `docker compose up -d`, mediante el cual se comprobó que los contenedores definidos en el fichero `docker-compose.yml` se crearon correctamente y quedaron en ejecución. Esta evidencia validó parcialmente los casos de prueba **CP-01** y **CP-02**.

Repositorio de código fuente en GitHub:

Como apoyo a la documentación técnica, todo el código desarrollado se centralizó en un repositorio de **GitHub**. En dicho repositorio se incluyeron los scripts Bash, los ficheros de Ansible, la configuración de Docker Compose y el resto de archivos necesarios para reproducir el entorno implementado.

El uso de GitHub permitió mantener trazabilidad sobre los cambios realizados y conservar versiones funcionales del despliegue (GitHub, s. f.).¹¹

Repositorio del proyecto: <https://github.com/Quontyx/IJ-Systems-Automation>

¹¹ Documentación oficial de GitHub sobre repositorios. Disponible en: <https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories>. Plataforma utilizada para alojar el código fuente del proyecto y mantener la trazabilidad de los cambios.



Figura 6.33 : Imagen del Repositorio GitHub.

Fuente: elaboración propia.

A continuación, se muestra el árbol de directorios del repositorio, donde se aprecia la separación entre scripts, Ansible, Docker, monitorización y documentación:

```
ubuntu@ip-172-31-36-8: ~/IJ-
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ tree
.
├── LICENSE
├── README.md
├── ansible
│   ├── group_vars
│   │   └── all.yml
│   ├── inventory.ini
│   └── playbook.yml
├── backups
│   ├── backup.log
│   └── backup_2026-05-25_19-30-30.sql
├── docker
│   ├── docker-compose.yml
│   └── nginx
│       ├── index.html
│       └── nginx.conf
├── docs
│   ├── logo.jpg
│   ├── pruebas
│   └── casos-prueba.md
├── logs
├── monitoring
│   ├── prometheus
│   └── prometheus.yml
└── scripts
    ├── backup.sh
    ├── checks.sh
    ├── instalar.sh
    └── restore.sh

12 directories, 17 files
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ |
```

Figura 6.34 : Estructura de directorios del repositorio del proyecto.

Fuente: elaboración propia.

6.7 Caso práctico completo de despliegue y validación en AWS

Planteamiento del caso práctico: una pequeña empresa o entorno de laboratorio necesita disponer, en menos de quince minutos y sin intervenciones manuales repetitivas, de un servidor Linux con varios servicios desplegados, supervisados y respaldados, accesible desde Internet con una política de seguridad básica y reproducible en cualquier otra cuenta de **AWS**. El caso práctico consistió en construir, automatizar y validar esa solución de extremo a extremo, partiendo de cero (sin infraestructura previa) y empleando únicamente los recursos descritos en la memoria. El resultado se considera válido si, al finalizar el procedimiento, los seis contenedores de la pila están en ejecución, **Nginx** responde por **HTTP** desde la **IP elástica**, los objetivos de **Prometheus** aparecen en estado UP, **Uptime Kuma** supervisa el servicio web, **cAdvisor** muestra métricas de los contenedores y **MariaDB** conserva los datos tras un reinicio y permite restaurar una copia de seguridad previamente generada.

El caso práctico se resolvió como una implantación completa en **AWS**. Se cubrió desde la creación de la infraestructura hasta la comprobación de los servicios y la recuperación de datos. Su objetivo fue demostrar que la arquitectura propuesta podía desplegarse y validarse de forma repetible.

El escenario partió de una instancia **Amazon EC2** con **Ubuntu Server**. La instancia contó con acceso mediante clave SSH, grupo de seguridad con reglas controladas e **IP elástica** asociada. A partir de ese punto, se preparó el host, se clonó el repositorio, se ejecutaron las automatizaciones y se levantó la pila de contenedores mediante **Docker Compose** (Amazon Web Services, s. f.-g; Docker Inc., s. f.-a).

La resolución del caso práctico se consideró correcta al cumplirse varias evidencias. **Nginx** respondió desde el navegador. **Prometheus** mostró sus objetivos en estado UP. **Uptime Kuma** supervisó el servicio HTTP. **cAdvisor** visualizó los contenedores. **MariaDB** conservó los datos tras el reinicio. Además, los scripts backup.sh y restore.sh permitieron copiar y recuperar la base de datos de prueba (Prometheus Authors, s. f.-d; Lam, s. f.; Google, s. f.; MariaDB plc, s. f.-b).

Fase	Acción realizada	Comando o evidencia	Resultado esperado
1. Infraestructura	Creación de EC2, clave SSH, grupo de seguridad e IP elástica.	Capturas de AWS y conexión SSH.	Servidor accesible y reglas de entrada controladas.
2. Preparación	Actualización del host e instalación de dependencias.	instalar.sh / playbook.yml	Docker, Docker Compose y Ansible disponibles.
3. Despliegue	Clonado del repositorio y arranque de la pila.	docker compose up -d	Contenedores en ejecución y redes/volúmenes creados.
4. Validación	Comprobación de Nginx, Prometheus, Uptime Kuma y cAdvisor.	Navegador, targets UP y docker ps.	Servicios accesibles y métricas visibles.
5. Persistencia	Prueba de MariaDB antes y después de reiniciar el contenedor.	INSERT, SELECT y docker restart.	El dato de prueba permanece disponible.
6. Recuperación	Copia y restauración de la base de datos de prueba.	backup.sh y restore.sh	Datos recuperados correctamente tras la restauración.

Tabla 6.6 : Desarrollo del caso práctico completo.

Fuente: elaboración propia.

Durante la ejecución se registraron incidencias reales. Entre ellas destacaron los permisos insuficientes sobre el socket de Docker, el acceso inicial incorrecto a MariaDB y los ajustes de puertos en el grupo de seguridad. Cada incidencia se documentó con su causa probable, la corrección aplicada y el resultado final obtenido.

Resolución del caso práctico: el caso práctico planteado al inicio de este apartado quedó resuelto de forma completa y sin errores pendientes. Se cumplieron los seis criterios de validez establecidos: la pila de seis contenedores arrancó correctamente con un único comando; **Nginx** respondió por **HTTP** desde la **IP elástica**; los objetivos de **Prometheus** aparecieron en estado UP; **Uptime Kuma** supervisó el servicio web; **cAdvisor** mostró métricas de los contenedores; y **MariaDB** conservó los datos tras el reinicio y permitió restaurar la copia previamente generada. El caso práctico no solo se planteó y explicó, sino que también se resolvió en condiciones reales, con las incidencias identificadas y corregidas durante el proceso.

7 Pruebas

El trabajo realizado permitió confirmar que la base técnica del proyecto quedó operativa. La estructura del repositorio se organizó correctamente, los ficheros clave se generaron, la pila se desplegó de forma reproducible y los servicios principales respondieron según lo previsto.

7.1 Casos de prueba

Con el trabajo realizado se completaron las pruebas funcionales principales sobre el entorno desplegado. En la tabla siguiente se resume el resultado obtenido en cada una de ellas.

ID	Prueba	Entrada o evidencia	Resultado obtenido	Estado
CP-01	Validación del fichero Compose	docker compose up -d	La pila arranca con 6/6 contenedores en ejecución.	Correcto
CP-02	Comprobación de contenedores activos	docker ps / docker compose ps	Todos los servicios principales aparecen levantados y con puertos esperados.	Correcto
CP-03	Acceso a la web de Nginx	http://<IP_PUBLICA_ELASTICA>	La página principal del proyecto carga correctamente desde navegador.	Correcto
CP-04	Targets de Prometheus	Status → Targets	Prometheus, node-exporter y cAdvisor aparecen en estado UP.	Correcto
CP-05	Supervisión en Uptime Kuma	http://<IP_PUBLICA_ELASTICA>:3001	El monitor configurado se muestra como funcional.	Correcto
CP-06	Acceso a cAdvisor	http://<IP_PUBLICA_ELASTICA>:8080	Se visualizan contenedores y métricas de recursos.	Correcto
CP-07	Ejecución básica de Ansible	ansible ping y playbook	La conectividad y la ejecución del playbook quedan validadas.	Correcto
CP-08	Persistencia de MariaDB	Crear base de datos, tabla, insertar registro, reiniciar contenedor y consultar de nuevo	El registro de prueba permanece disponible tras el reinicio, validando el volumen persistente.	Correcto

ID	Prueba	Entrada o evidencia	Resultado obtenido	Estado
CP-09	Copia de seguridad de MariaDB	./backup.sh	Se genera un fichero .sql de copia de seguridad en el directorio de backups.	Correcto
CP-10	Restauración de MariaDB	./restore.sh nombre_del_backup.sql	La base de datos puede restaurarse a partir del fichero generado.	Correcto
CP-11	Validación posterior a restauración	Consulta SQL posterior a restore	Los datos restaurados pueden consultarse correctamente.	Correcto
CP-12	Trazabilidad del repositorio	git status / estructura del repositorio	El código queda centralizado y organizado en GitHub con separación por carpetas.	Correcto
CP-13	Automatización del backup mediante cron	crontab -l y fichero backup.log generado	La tarea queda registrada en crontab y el script genera el fichero .sql con timestamp en backups/ sin intervención manual	Correcto
CP-14	Acceso a servicios de monitorización a través del proxy inverso Nginx	http://<IP>/prometheus/ y http://<IP>/metricas/	Prometheus accesible en /prometheus/ y cAdvisor en /metricas/ con HTTP 200, sin exponer los puertos directamente	Correcto

Tabla 7.1 : Casos de prueba.

Fuente: elaboración propia.

Además de la tabla de pruebas, se relacionaron las evidencias gráficas y técnicas utilizadas para justificar los resultados obtenidos.

Evidencia	Qué demuestra
Instancia EC2 creada	La infraestructura base en AWS quedó desplegada correctamente.
Grupo de seguridad	Se aplicaron reglas de acceso diferenciadas para administración, web y monitorización.
Conexión SSH	El acceso administrativo se realizó mediante clave privada.
docker compose up -d	El despliegue de la pila se ejecutó de forma automatizada.
docker ps / docker compose ps	Los contenedores principales quedaron en ejecución.
Nginx en navegador	El servicio web publicado respondió desde la IP pública.
Prometheus Targets UP	La monitorización recogió métricas de los objetivos configurados.
Uptime Kuma	La supervisión externa de disponibilidad quedó configurada.
cAdvisor	Se visualizaron métricas de uso de contenedores.
MariaDB tras reinicio	La persistencia de datos quedó validada mediante volumen.
Archivo: backup.sh	Generación correcta de una copia de seguridad de MariaDB en formato .sql.

Evidencia	Qué demuestra
Archivo: restore.sh	Restauración funcional de la base de datos a partir de una copia previamente generada.
Consulta SQL tras restore	Los datos restaurados quedan disponibles y accesibles tras el proceso de recuperación.
git status / git log / estructura del repositorio	Trazabilidad de cambios, control de versiones y organización del código fuente del proyecto.

Tabla 7.2 : Relación de evidencias utilizadas en la validación.

Fuente: elaboración propia.

La siguiente comparación resume la mejora conseguida frente a un proceso completamente manual. Los tiempos automatizados proceden de las ejecuciones observadas durante las pruebas de despliegue, copia y restauración. Los tiempos manuales son estimaciones razonadas para ejecutar las mismas operaciones comando a comando en un entorno de laboratorio. No se realizó un benchmark cronometrado formal, porque el objetivo de la prueba era validar reproducibilidad, reducción de errores y mejora operativa, no medir rendimiento de hardware. Aun así, el rango agregado permite comparar el impacto: de unos 60-90 minutos manuales se pasó a aproximadamente 8-18 minutos con automatización, sin contar la trazabilidad aportada por GitHub.

Tarea	Proceso manual estimado	Proceso automatizado	Mejora obtenida
Preparación de dependencias	25-35 min instalando repositorios, paquetes y comprobaciones manualmente.	5-10 min con instalar.sh / playbook.yml.	Menos omisiones y configuración repetible.
Despliegue de servicios	15-20 min arrancando y validando servicios por separado.	1-3 min con docker compose up -d.	Arranque centralizado de toda la pila.
Comprobación del estado	10-15 min revisando contenedores, puertos y logs.	1-2 min con checks.sh y docker compose ps.	Validación más rápida y homogénea.
Copia de seguridad	5-10 min ejecutando comandos MariaDB manuales.	Menos de 1 min con backup.sh.	Procedimiento estandarizado y repetible.
Restauración	5-10 min importando manualmente el fichero SQL.	1-2 min con restore.sh.	Recuperación más guiada y con menor riesgo de error.
Trazabilidad	Difícil de cuantificar: cambios dispersos o no documentados.	Registro en GitHub mediante commits.	Historial revisable y control de versiones.
Resultado agregado	60-90 min en tareas operativas comparables.	8-18 min en tareas automatizadas comparables.	Reducción aproximada del 70%-90% del tiempo operativo y menor dependencia de intervención manual.

Tabla 7.3 : Comparativa entre proceso manual y proceso automatizado.

Fuente: elaboración propia.

IJ Systems Automation — Administración de Sistemas Informáticos en Red

Esta comparación no pretende sustituir una auditoría de rendimiento formal. Su finalidad es justificar, con una metodología transparente, la mejora práctica obtenida al automatizar operaciones repetitivas y reducir la intervención manual en el despliegue.

Las capturas siguientes recogen las principales evidencias técnicas obtenidas durante la validación del proyecto. En conjunto, permiten comprobar el funcionamiento de la publicación web, la monitorización, la supervisión de disponibilidad, la persistencia de datos, los procedimientos de copia y restauración y la trazabilidad del repositorio.

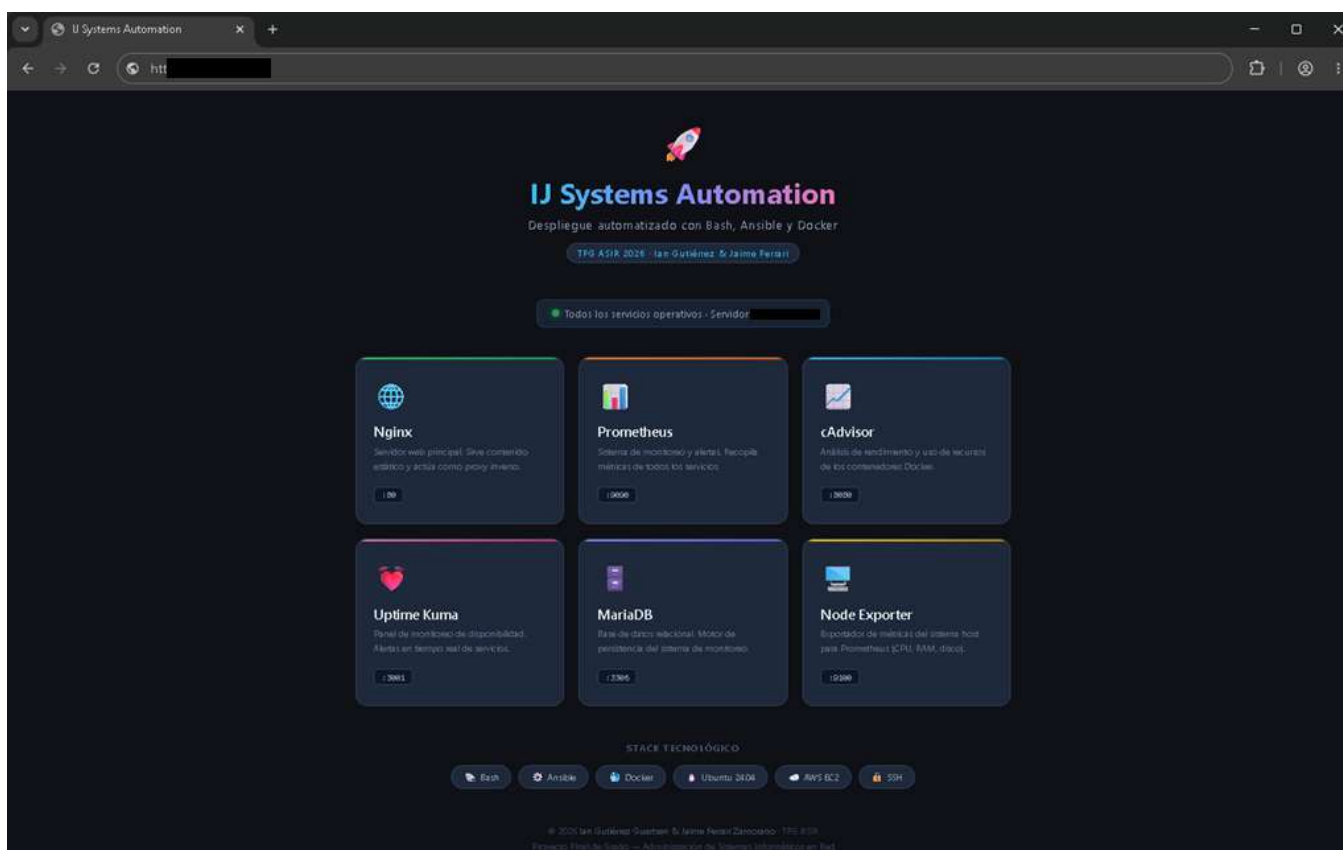


Figura 7.1 : Validación del servicio web Nginx desde navegador.

Fuente: elaboración propia.

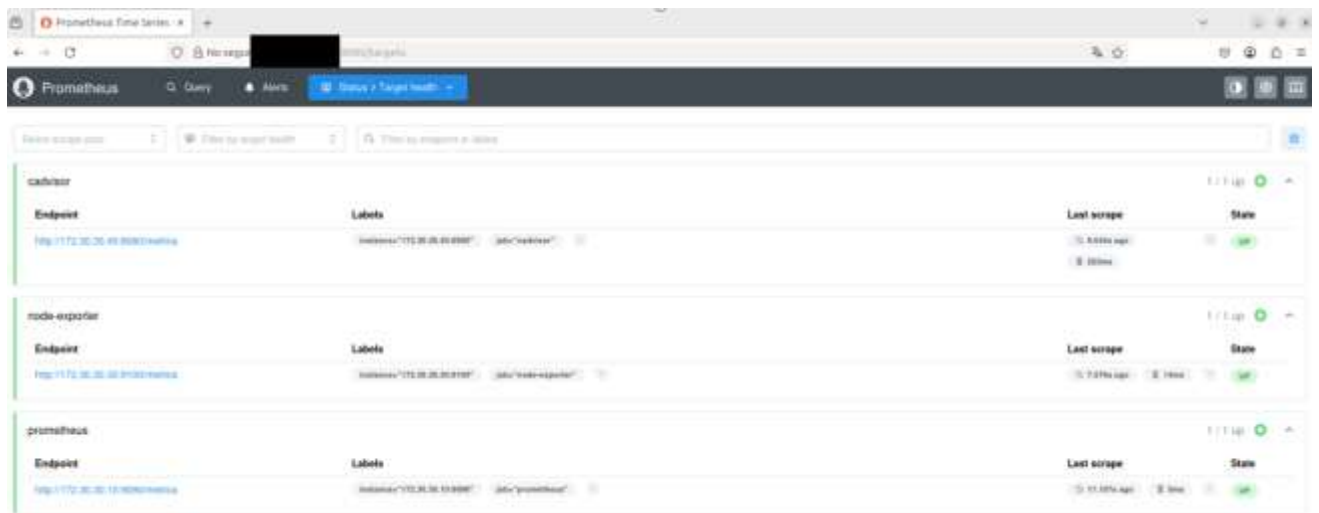


Figura 7.2 : Validación de objetivos activos en Prometheus.

Fuente: elaboración propia.



Figura 7.3 : Supervisión del servicio HTTP en Uptime Kuma.

Fuente: elaboración propia.

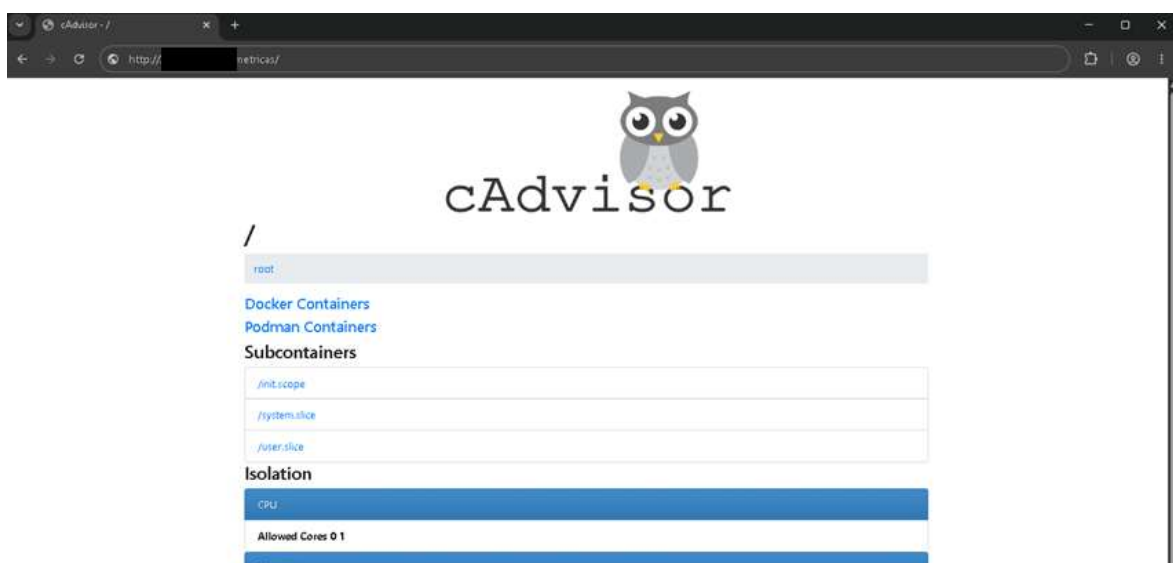


Figura 7.4 : Visualización de contenedores Docker en cAdvisor.

Fuente: elaboración propia.

```
MariaDB [ijdb]> DROP TABLE IF EXISTS prueba;
Query OK, 0 rows affected (0.009 sec)

MariaDB [ijdb]> CREATE TABLE prueba (
  -> id INT AUTO_INCREMENT PRIMARY KEY,
  -> nombre VARCHAR(50)
  -> );
Query OK, 0 rows affected (0.009 sec)

MariaDB [ijdb]> INSERT INTO prueba (nombre) VALUES ('dato1');
Query OK, 1 row affected (0.002 sec)

MariaDB [ijdb]> SELECT * FROM prueba;
+----+-----+
| id | nombre |
+----+-----+
| 1  | dato1  |
+----+-----+
1 row in set (0.001 sec)

MariaDB [ijdb]>
```

Figura 7.5 : Validación limpia de inserción y consulta de datos en MariaDB.

Fuente: elaboración propia.

```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/scripts$ ./backup.sh
Backup realizado correctamente: /home/ubuntu/IJ-Systems-Automation/backups/backup_2026-04-25_14-18-08.sql
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/scripts$ cd ..
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation$ cd backups/
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/backups$ ls
backup_2026-04-25_14-18-08.sql
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/backups$
```

Figura 7.6 : Ejecución correcta del script de copia de seguridad.

Fuente: elaboración propia.

```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/scripts$ docker exec -it mariadb mariadb -u root -p
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 15
Server version: 12.2.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE ijdb;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [ijdb]> DROP TABLE prueba;
Query OK, 0 rows affected (0.008 sec)

MariaDB [ijdb]> SHOW TABLES;
Empty set (0.000 sec)

MariaDB [ijdb]> EXIT;
Bye
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/scripts$
```

Figura 7.7 : Eliminación de la tabla de prueba antes de la restauración.

Fuente: elaboración propia.


```
ubuntu@ip-172-31-36-8: ~/IJ- x + v
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$ ./restore.sh backup_2026-04-25_14-18-08.sql
Restauración completada correctamente
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$
```

Figura 7.8 : Restauración correcta de MariaDB mediante restore.sh.

Fuente: elaboración propia.

```
ubuntu@ip-172-31-36-8: ~/IJ- x + v
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$ docker exec -it mariadb mariadb -u root -p
Welcome to the MariaDB monitor.  Commands end with ; or \g.
Your MariaDB connection id is 18
Server version: 12.2.2-MariaDB-ubu2404 mariadb.org binary distribution

Copyright (c) 2000, 2018, Oracle, MariaDB Corporation Ab and others.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

MariaDB [(none)]> USE ijdbc;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
MariaDB [ijdbc]> SHOW TABLES;
+-----+
| Tables_in_ijdbc |
+-----+
| prueba          |
+-----+
1 row in set (0.000 sec)

MariaDB [ijdbc]> SELECT * FROM prueba;
+----+-----+
| id | nombre |
+----+-----+
| 1  | dato1  |
+----+-----+
1 row in set (0.000 sec)

MariaDB [ijdbc]> EXIT;
Bye
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/scripts$
```

Figura 7.9 : Validación de datos restaurados en MariaDB.

Fuente: elaboración propia.

```

ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ git log --oneline -5
8d6102f (HEAD -> main) Mejorar index de Nginx: nuevo dashboard moderno con tarjetas de servicios
2807874 (origin/main, origin/HEAD) Mejorar scripts, pinear versiones Docker y añadir health checks
f521378 Actualizar casos de prueba y checks: sustituir Grafana por Uptime Kuma
8b86d0d Eliminar copias locales con credenciales y reforzar gitignore
13790ea Eliminar credenciales reales de scripts y usar variables de entorno
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ tree -L 2
.
├── README.md
├── ansible
│   ├── group_vars
│   ├── inventory.ini
│   └── playbook.yml
├── backups
│   ├── backup.log
│   └── backup_2026-05-25_19-30-30.sql
├── docker
│   ├── docker-compose.yml
│   └── nginx
├── docs
│   └── pruebas
├── logs
├── monitoring
│   └── prometheus
├── scripts
│   ├── backup.sh
│   ├── checks.sh
│   ├── instalar.sh
│   └── restore.sh
└── 12 directories, 10 files
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$

```

Figura 7.10 : Trazabilidad del repositorio y estructura final del proyecto.

Fuente: elaboración propia.

8 Explotación

La explotación del sistema se planteó para un escenario de pequeña empresa o laboratorio técnico. En esta fase no se repite la implementación, sino que se define cómo operar el entorno una vez desplegado: arranque, validación, mantenimiento básico, control de cambios y recuperación ante incidencias.

8.1 Planificación

La planificación de explotación se basó en una secuencia estable y repetible. Primero se comprobó que la instancia estuviera disponible y que el acceso **SSH** funcionara. Después se verificó que el repositorio y los ficheros de configuración se encontraran en la versión correcta. Por último, se ejecutaron las comprobaciones de estado y las pruebas básicas de servicio.

Los recursos necesarios fueron reducidos: una instancia **EC2 con Ubuntu**, conectividad a Internet, clave **SSH**, acceso al repositorio **GitHub** y permisos para modificar reglas de red en caso de incidencia. El equipo responsable de la explotación inicial estuvo formado por los autores del proyecto.

Los riesgos operativos más importantes fueron la pérdida de conectividad **SSH**, una regla de red mal aplicada, un fallo de arranque de contenedores o una restauración incorrecta de

datos. Para reducirlos, se definió una validación por etapas y se mantuvo un registro de incidencias.

8.2 Preparación para el cambio

Antes de aplicar cambios, se comprobó que existiera una versión funcional del repositorio y que los ficheros sensibles estuvieran correctamente gestionados. También se revisó que la **IP elástica** y las reglas de entrada fueran coherentes con los servicios que se iban a validar.

Para minimizar errores, cada cambio debía aplicarse de forma ordenada y quedar registrado. Esta pauta permitió volver a una versión anterior del código o repetir una comprobación sin depender de instrucciones improvisadas.

8.3 Plan de formación

La formación necesaria para explotar el sistema fue básica. Se centró en acceder al host mediante **SSH**, interpretar el estado de los contenedores, ejecutar los **scripts** de comprobación, realizar copias de seguridad y consultar las herramientas de monitorización. El anexo de operación recoge los comandos mínimos para estas tareas.

8.4 Implantación propiamente dicha

La implantación se consideró completada cuando el entorno pudo operarse sin pasos manuales no documentados. Para ello, debían estar disponibles el acceso **SSH**, el repositorio, la pila de contenedores, los servicios web y las pruebas funcionales básicas.

8.5 Pruebas de implantación

Las pruebas de implantación se orientaron a confirmar que el entorno seguía siendo utilizable después del despliegue. Se revisó la publicación web, el estado de los objetivos de monitorización, la supervisión de disponibilidad, la persistencia de MariaDB y la ejecución de los procedimientos de copia y restauración.

8.6 Paso a un entorno productivo

El alcance del proyecto se ajustó deliberadamente a un escenario académico, de laboratorio o de pequeña empresa, con una infraestructura ligera y un uso puntual de los recursos de AWS. En este apartado se recogen, de forma estructurada, las decisiones y ajustes que serían necesarios si la misma solución se llevase a un entorno de producción real. No se trata de modificaciones que invaliden el resultado obtenido, sino de las líneas naturales de evolución que delimitan con precisión el alcance actual del trabajo y permiten proyectar la solución hacia una implantación profesional.

Dimensionamiento de la infraestructura y costes. Si en futuras ampliaciones se incorporasen nuevos servicios o una arquitectura más compleja, sería necesario revisar el dimensionamiento de la instancia, la política de copias de seguridad, la gestión de credenciales y los costes asociados a monitorización, almacenamiento y transferencia de datos. La estimación de 13 USD/mes presentada en el apartado 3.7.1 corresponde a la

configuración mínima evaluada (t3.micro, 20 GB EBS gp3 e IPv4 pública en funcionamiento continuo) y no incluye impuestos, posibles variaciones por región, tráfico de salida elevado, snapshots, dominios ni certificados externos. Para una estimación más precisa en un entorno real, se recomienda utilizar AWS Pricing Calculator, ya que permite modelar servicios y planificar el gasto según los recursos seleccionados (Amazon Web Services, s. f.-e).¹²

Gestión de secretos. La gestión de credenciales mediante .env y .gitignore aplicada en el proyecto es suficiente para un entorno académico, pero en un escenario corporativo debería sustituirse por una solución específica de gestión de secretos, como HashiCorp Vault, AWS Secrets Manager o equivalente. En la misma línea, las contraseñas no deberían escribirse en texto claro dentro de comandos, documentación, scripts o repositorios. Para accesos puntuales a servicios como MariaDB se recomienda introducir la contraseña de forma interactiva con la opción -p del cliente, evitando que quede registrada en el historial de la consola.

Publicación segura con HTTPS. En el proyecto se publicó el servicio web mediante HTTP por el puerto 80, suficiente para validar funcionalmente la pila desde un navegador. El puerto 443 quedó contemplado como ampliación natural para HTTPS, pero no formó parte de la validación funcional principal de esta versión. En un entorno productivo, la publicación del servicio debería realizarse con certificados TLS, gestionables mediante una autoridad como Let's Encrypt con Certbot o un certificado corporativo equivalente. Esta evolución implicaría también ajustar el grupo de seguridad para habilitar el puerto 443 y revisar la configuración del servidor web para soportar la terminación TLS.

Copias de seguridad programadas y monitorización con alertas. El proyecto incorporó los scripts backup.sh y restore.sh y dejó configurada una tarea programada mediante cron para automatizar la copia de seguridad de MariaDB. Esta solución fue suficiente para validar el procedimiento en un entorno académico. En un entorno productivo convendría ampliar esta automatización con una política de rotación, retención, cifrado y almacenamiento externo de respaldos, evitando el crecimiento indefinido del directorio de copias. De forma análoga, la monitorización implantada recoge métricas del host y de los contenedores, pero no incorpora alertas automáticas. En producción sería recomendable añadir un componente de alertas, como Alertmanager integrado con Prometheus, configurado con reglas básicas —servicio caído, uso elevado de disco o memoria, ausencia prolongada de métricas— y un canal de notificación.

Ampliación del stack de observabilidad y operación. Durante el diseño se descartaron de forma deliberada herramientas adicionales como Grafana o Portainer, con el fin de mantener el alcance del proyecto ajustado a una implantación académica, ligera y sencilla. En un entorno productivo, una vez consolidada la base actual, sería razonable incorporar paneles de visualización en Grafana sobre los datos ya expuestos por Prometheus, así como una herramienta de gestión gráfica de contenedores como Portainer, que facilita la operación a equipos con perfiles menos técnicos. Estas incorporaciones no sustituirían a la arquitectura tripartita Bash + Ansible + Docker Compose, sino que se apoyarían sobre ella.

¹² Calculadora oficial de precios de AWS (AWS Pricing Calculator). Disponible en: <https://calculator.aws/>.
Recomendada para realizar estimaciones precisas de coste mensual fuera del entorno AWS Academy.

En conjunto, estas cinco líneas de evolución (dimensionamiento, secretos, HTTPS, copias y alertas programadas, y ampliación de la observabilidad) no forman parte del alcance actual, pero permiten proyectar la solución hacia un entorno más cercano a una implantación profesional manteniendo intacta la base técnica entregada.

9 Definición de procedimientos de control y evaluación

A lo largo del ciclo de vida del proyecto podían producirse cambios e incidencias que debían controlarse y registrarse. Para ello, se definió un procedimiento sencillo, pero formal, que permitió documentar el problema, su causa probable, la acción correctiva aplicada y el resultado final obtenido.

Cada incidencia detectada durante la implantación, las pruebas o la explotación se registró con una estructura homogénea, lo que redujo el tiempo de diagnóstico y permitió relacionar cada error con su evidencia.

Campo	Descripción
Fecha y versión	Momento de detección y versión del repositorio o despliegue afectado.
Identificación de la incidencia	Título breve del problema o cambio detectado.
Descripción del problema	Contexto en el que se produce y síntoma observado.
Resultado esperado	Comportamiento que debía producirse según el diseño o la prueba.
Resultado obtenido	Comportamiento real observado durante la validación.
Causa probable	Hipótesis técnica que explica el error detectado.
Acción correctiva	Medida aplicada o propuesta para resolver la incidencia.
Estado final	Correcto, parcial, pendiente o descartado.
Área afectada	Configuración, scripts Bash, Ansible, Docker Compose, servicios o documentación.

Tabla 9.1 : Plantilla de registro de incidencias y control de cambios.

Fuente: elaboración propia.

Además del control de incidencias, cualquier cambio funcional importante quedó reflejado en **GitHub** mediante commits descriptivos, de forma que existiera trazabilidad técnica entre la memoria y el código fuente (GitHub, s. f.).

Como ejemplo de este método, en el proyecto se registraron y corrigieron incidencias relacionadas con la autenticación en GitHub, los permisos del socket de Docker, el acceso SSH, el acceso a MariaDB, la persistencia de datos y el ajuste de puertos en el grupo de seguridad.

10 Conclusiones

Las tareas operativas comparables que en un proceso manual exigían entre 60 y 90 minutos pasaron a ejecutarse, una vez automatizadas, en un intervalo aproximado de 8 a 18 minutos. Esto supone una reducción operativa cercana al 70 %-90 %, además de la trazabilidad aportada por el repositorio versionado en GitHub. La automatización redujo la dependencia de operaciones manuales y dejó una base reutilizable para entornos de laboratorio o pequeñas implantaciones. La solución obtenida respondió a la necesidad inicial de reducir tiempos de implantación, minimizar errores humanos y disponer de una base técnica reutilizable.

El proyecto permitió diseñar e implantar una solución funcional de automatización basada en Bash, Ansible y Docker sobre una instancia Ubuntu en AWS. Desde el punto de vista técnico, los objetivos principales se consideraron cumplidos: preparar el servidor, automatizar tareas repetitivas, encapsular servicios, documentar el proceso y comprobar el funcionamiento del entorno en un escenario realista. Desde el punto de vista formativo, el proyecto reforzó conocimientos de administración Linux, contenedores, automatización, control de versiones y despliegue en la nube. Desde una perspectiva crítica, la decisión más acertada fue separar responsabilidades entre Bash, Ansible y Docker Compose. Esta separación evitó que todo el proyecto dependiera de un único script difícil de mantener.

Como limitaciones del trabajo, quedaron fuera del alcance la orquestación avanzada con Kubernetes, la gestión multiinstancia y una explotación corporativa completa. Estas limitaciones no invalidan el resultado obtenido, sino que delimitan con precisión el alcance real del trabajo y trazan las líneas naturales de evolución hacia un entorno productivo.

Si se contrastan los objetivos planteados al inicio del proyecto con los resultados obtenidos, el balance es positivo en los cinco objetivos SMART definidos (OBJ-01 a OBJ-05). La preparación del servidor Ubuntu quedó automatizada en una sola ejecución; el despliegue de la pila se realizó en menos de tres minutos; el servicio web fue validado desde navegador con éxito; la persistencia y recuperación de MariaDB se verificaron mediante volumen, backup y restore; y la trazabilidad quedó garantizada por el repositorio.

Por último, conviene señalar que un proyecto técnico no termina cuando los servicios responden, sino cuando otra persona puede repetir el despliegue sin asistencia. Cualquier usuario técnico que clone el repositorio, lea el manual básico de usuario incluido como anexo y siga el procedimiento de despliegue podrá reconstruir el entorno desde cero y obtener resultados equivalentes. Esa repetibilidad es la que diferencia un experimento puntual de una solución de administración de sistemas mantenible.

11 Glosario

En este apartado se recogen los principales tecnicismos y acrónimos utilizados en la memoria, con el fin de facilitar la lectura.

AWS: Amazon Web Services, plataforma de servicios en la nube utilizada para alojar la instancia del proyecto.

EC2: Elastic Compute Cloud, servicio de AWS utilizado para crear la máquina virtual Ubuntu donde se desplegó el entorno.

SSH: Secure Shell, protocolo empleado para acceder de forma remota y segura al servidor.

Bash: intérprete de comandos utilizado para automatizar tareas repetitivas del sistema.

Ansible: herramienta de automatización declarativa usada para preparar y configurar el host.

Docker: plataforma de contenedores empleada para ejecutar los servicios del proyecto de forma aislada.

Docker Compose: herramienta utilizada para definir y levantar la pila de servicios mediante un único fichero de configuración.

Nginx: servidor web utilizado para publicar la página de validación del proyecto.

MariaDB: sistema gestor de base de datos utilizado para validar persistencia, copias y restauración.

Prometheus: sistema de monitorización utilizado para recoger métricas del host y de los servicios configurados.

node-exporter: exportador de métricas del host consultado por Prometheus.

cAdvisor: herramienta utilizada para visualizar métricas de consumo de los contenedores Docker.

Uptime Kuma: herramienta de supervisión de disponibilidad utilizada para comprobar el estado del servicio web.

GitHub: plataforma de control de versiones empleada para alojar el repositorio del proyecto.

IP elástica: dirección IPv4 pública persistente asociada a la instancia EC2 durante las pruebas.

Volumen Docker: mecanismo de persistencia utilizado para conservar datos, aunque se reinicie o recree un contenedor.

Playbook: fichero en formato YAML donde Ansible define las tareas declarativas a ejecutar sobre uno o varios hosts del inventario.

Inventario: archivo donde Ansible declara los hosts gestionados y su organización en grupos.

YAML: formato de serialización de datos legible por humanos, utilizado por Ansible y Docker Compose para definir configuraciones.

Socket: punto de comunicación entre procesos. En el contexto del proyecto, el socket de Docker (/var/run/docker.sock) permite al cliente comunicarse con el demonio del motor de contenedores.

VPC: Virtual Private Cloud, red virtual aislada dentro de AWS donde se despliegan los recursos de la cuenta.

AMI: Amazon Machine Image, plantilla de máquina virtual a partir de la cual se lanza una instancia EC2 en AWS.

CIDR: Classless Inter-Domain Routing, notación para representar rangos de direcciones IP, utilizada en las reglas de seguridad de AWS.

TLS / HTTPS: Transport Layer Security y HyperText Transfer Protocol Secure, protocolo y combinación usados para cifrar comunicaciones web. Contemplados como ampliación futura del proyecto.

CFGS: Ciclo Formativo de Grado Superior, dentro del cual se enmarca el título de Administración de Sistemas Informáticos en Red (ASIR).

TFG: Trabajo de Fin de Grado o, en este contexto, Proyecto de Fin de Ciclo presentado al final del CFGS de ASIR.

DevOps: conjunto de prácticas que combinan desarrollo (Dev) y operaciones (Ops), basado en automatización, despliegue continuo y colaboración entre equipos. La arquitectura del proyecto refleja varios de estos principios.

12 Bibliografía

- Amazon Web Services. (s. f.-a). Amazon EBS pricing.
<https://aws.amazon.com/ebs/pricing/>
- Amazon Web Services. (s. f.-b). Amazon EC2 On-Demand Instance Pricing.
<https://aws.amazon.com/ec2/pricing/on-demand/>
- Amazon Web Services. (s. f.-c). Amazon EC2 security groups for your EC2 instances.
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ec2-security-groups.html>
- Amazon Web Services. (s. f.-d). Amazon VPC Pricing.
<https://aws.amazon.com/vpc/pricing/>
- Amazon Web Services. (s. f.-e). AWS Pricing Calculator.
<https://calculator.aws/>
- Amazon Web Services. (s. f.-f). Connect to your Linux instance using SSH.
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/connect-to-linux-instance.html>
- Amazon Web Services. (s. f.-g). What is Amazon EC2?
<https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/concepts.html>
- Ansible Community. (s. f.-a). Ansible documentation.
<https://docs.ansible.com/projects/ansible/latest/index.html>
- Ansible Community. (s. f.-b). Ansible playbooks.
https://docs.ansible.com/projects/ansible/latest/playbook_guide/playbooks_intro.html
- Ansible Community. (s. f.-c). How to build your inventory.
https://docs.ansible.com/projects/ansible/latest/inventory_guide/intro_inventory.html
- Docker Inc. (s. f.-a). Docker Compose documentation.
<https://docs.docker.com/compose/>
- Docker Inc. (s. f.-b). Docker Compose quickstart.
<https://docs.docker.com/compose/gettingstarted/>
- Docker Inc. (s. f.-c). Docker Engine installation on Ubuntu.
<https://docs.docker.com/engine/install/ubuntu/>
- GitHub. (s. f.). About repositories.
<https://docs.github.com/en/repositories/creating-and-managing-repositories/about-repositories>
- Google. (s. f.). cAdvisor.
<https://github.com/google/cadvisor>
- Lam, L. (s. f.). Uptime Kuma.
<https://github.com/louislam/uptime-kuma>
- MariaDB plc. (s. f.-a). MariaDB command-line client.
<https://mariadb.com/docs/server/clients-and-utils/mariadb-client/mariadb-command-line-client>

MariaDB plc. (s. f.-b). mariadb-dump.

<https://mariadb.com/docs/server/clients-and-utilities/backup-restore-and-import-clients/mariadb-dump>

Nginx. (s. f.). Nginx documentation.

<https://nginx.org/en/docs/>

Prometheus Authors. (s. f.-a). Configuration.

<https://prometheus.io/docs/prometheus/latest/configuration/configuration/>

Prometheus Authors. (s. f.-b). Getting started.

https://prometheus.io/docs/prometheus/latest/getting_started/

Prometheus Authors. (s. f.-c). Monitoring Linux host metrics with the Node Exporter.

<https://prometheus.io/docs/guides/node-exporter/>

Prometheus Authors. (s. f.-d). Prometheus overview.

<https://prometheus.io/docs/introduction/overview/>

Ubuntu. (s. f.). Ubuntu Server documentation.

<https://documentation.ubuntu.com/server/>

13 Anexos

En este apartado se recopila la documentación complementaria que amplía la memoria principal y facilita la explotación y el mantenimiento del proyecto. Los anexos no sustituyen al contenido técnico desarrollado en la memoria, sino que lo apoyan con información práctica sobre operación, planificación y resolución de incidencias.

Estos anexos dejan constancia del modo de uso básico del entorno, del orden recomendado de implantación y de los problemas más relevantes detectados durante el desarrollo, facilitando así la operación y reproducción posterior del proyecto.

13.1 Manual básico de usuario

El presente anexo recoge las operaciones mínimas necesarias para utilizar y comprobar el entorno desplegado. Está orientado a un usuario técnico con conocimientos básicos de Linux, Docker y acceso remoto mediante SSH.

13.1.1 Acceso al servidor

El acceso al entorno se realiza mediante **SSH** sobre la instancia **Amazon EC2** con Ubuntu, utilizando el archivo de clave privada generado durante la creación de la instancia (Amazon Web Services, s. f.-f).

Comando general de acceso:

```
ssh -i "ruta/de/la/clave.pem" ubuntu@<IP_PUBLICA_ELASTICA>
```

Una vez establecida la conexión, el proyecto queda disponible en el directorio de trabajo correspondiente.

13.1.2 Servicios disponibles

Tras el despliegue correcto del entorno, los servicios principales quedaron accesibles mediante la **IP pública elástica** asociada a la instancia:

- **Nginx:** `http://<IP_PUBLICA_ELASTICA>`
- **Prometheus:** `http://<IP_PUBLICA_ELASTICA>/prometheus/`
- **Uptime Kuma:** `http://<IP_PUBLICA_ELASTICA>:3001`
- **cAdvisor:** `http://<IP_PUBLICA_ELASTICA>/metrics/`

MariaDB no se expuso públicamente. Su acceso se realizó únicamente desde el contenedor o desde la red interna del entorno Docker.

13.1.3 Puesta en marcha del entorno

Para levantar la pila completa de servicios, se ejecuta el siguiente comando desde el directorio `docker/` del proyecto:

```
docker compose up -d
```

Para detener la pila de servicios:

```
docker compose down
```

Para reiniciar los servicios:

```
docker compose restart
```

13.1.4 Comprobación del estado

El estado de los contenedores puede comprobarse mediante:

```
docker ps
```

o bien:

```
docker compose ps
```

Asimismo, el proyecto dispone de un script de comprobación que centraliza varias validaciones básicas del entorno:

```
./scripts/checks.sh
```

13.1.5 Copia de seguridad

La copia de seguridad de la base de datos **MariaDB** se realiza mediante el script:

`./scripts/backup.sh`

El resultado se almacena en el directorio destinado a copias de seguridad, con un nombre basado en la fecha y hora de ejecución.

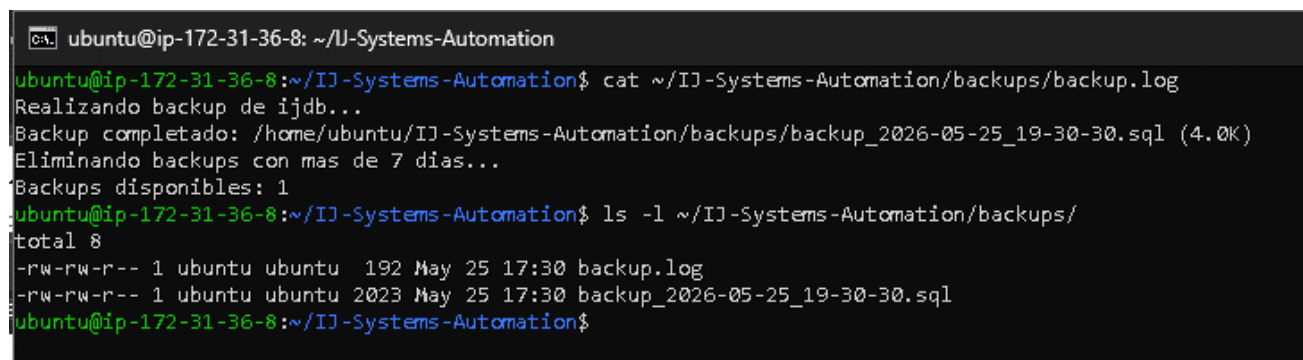
Para evitar la dependencia de una ejecución manual, la copia de seguridad se automatizó mediante una tarea programada con cron. Se añadió una entrada al crontab del usuario que invoca periódicamente el script `backup.sh`, de modo que la copia de la base de datos se genera de forma desatendida y queda registrada en el directorio de respaldos con su marca temporal correspondiente. La tarea programada puede consultarse con el comando `crontab -l`, tal como se muestra en la Figura 13.1.



```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation/ansible/group_vars$ crontab -l
0 2 * * * /home/ubuntu/IJ-Systems-Automation/scripts/backup.sh >> /home/ubuntu/IJ-Systems-Automation/backups/backup.log 2>&1
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation/ansible/group_vars$
```

Figura 13.1 : Configuración de la tarea programada de backup mediante cron.

Fuente: elaboración propia.



```
ubuntu@ip-172-31-36-8: ~/IJ-Systems-Automation
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ cat ~/IJ-Systems-Automation/backups/backup.log
Realizando backup de ijdbc...
Backup completado: /home/ubuntu/IJ-Systems-Automation/backups/backup_2026-05-25_19-30-30.sql (4.0K)
Eliminando backups con mas de 7 dias...
Backups disponibles: 1
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$ ls -l ~/IJ-Systems-Automation/backups/
total 8
-rw-rw-r-- 1 ubuntu ubuntu 192 May 25 17:30 backup.log
-rw-rw-r-- 1 ubuntu ubuntu 2023 May 25 17:30 backup_2026-05-25_19-30-30.sql
ubuntu@ip-172-31-36-8:~/IJ-Systems-Automation$
```

Figura 13.2 : Ejecución del backup automático y fichero de copia generado.

Fuente: elaboración propia.

13.1.6 Restauración

La restauración de una copia existente se realiza mediante el script:

`./scripts/restore.sh nombre_del_backup.sql`

13.1.7 Validación de MariaDB

Para acceder a **MariaDB** desde el contenedor, se utiliza el siguiente comando:

`docker exec -it mariadb mariadb -u root -p`

Desde la consola de MariaDB pueden ejecutarse consultas de comprobación, validación o mantenimiento sobre la base de datos del proyecto.

13.1.8 Recomendaciones de uso

- No modificar directamente los ficheros de configuración sin registrar previamente el cambio en el repositorio.
 - Validar el estado de los contenedores después de cada modificación importante.
 - Mantener restringido el acceso mediante **SSH** a direcciones IP autorizadas.
 - Utilizar los scripts del proyecto siempre que sea posible, en lugar de repetir comandos manuales.
-

13.2 Planificación operativa resumida

Este anexo resume el orden recomendado para implantar el proyecto en un entorno nuevo. Su finalidad es reducir errores, mantener la trazabilidad del procedimiento y asegurar que el despliegue se realice de forma homogénea.

13.2.1 Fase 1. Preparación de infraestructura

En esta fase se realiza la creación de la instancia **Amazon EC2**, la selección del sistema operativo **Ubuntu Server**, la generación del par de claves y la configuración inicial del grupo de seguridad. También se asocia la **IP pública elástica** utilizada durante las pruebas y la validación externa.

Resultado esperado: instancia accesible mediante **SSH**, con reglas de red correctamente configuradas.

13.2.2 Fase 2. Acceso y preparación del host

Se accede al servidor mediante **SSH**, se actualiza el sistema base y se comprueba la conectividad general. A continuación, se instalan **Docker**, **Docker Compose**, **Ansible** y las utilidades necesarias para la ejecución del proyecto.

Resultado esperado: host preparado para recibir el repositorio y ejecutar las automatizaciones del proyecto.

13.2.3 Fase 3. Despliegue del repositorio

Se clona el repositorio del proyecto, se revisa la estructura de directorios y se comprueba la disponibilidad de los scripts Bash, los ficheros de Ansible, las configuraciones de Docker Compose y los elementos de monitorización.

Resultado esperado: proyecto desplegado en el servidor con una estructura íntegra, organizada y versionada.

13.2.4 Fase 4. Levantamiento de servicios

Se ejecuta el fichero `docker-compose.yml` para levantar **Nginx**, **MariaDB**, **Prometheus**, **Uptime Kuma**, **cAdvisor** y el resto de servicios auxiliares definidos en la pila.

Resultado esperado: contenedores en ejecución, redes creadas y volúmenes persistentes preparados.

13.2.5 Fase 5. Validación funcional

Se comprueba la disponibilidad de los servicios desde el navegador, el estado de los objetivos en **Prometheus**, la supervisión configurada en **Uptime Kuma** y la visualización de recursos en **cAdvisor**. También se valida **MariaDB** mediante consola y consultas SQL de prueba.

Resultado esperado: entorno funcional y accesible desde la IP pública elástica.

13.2.6 Fase 6. Explotación básica y mantenimiento

Se ejecutan los scripts de comprobación, copia de seguridad y restauración, y se documentan las incidencias o cambios relevantes detectados. Esta fase asegura que el entorno no solo arranca correctamente, sino que también puede mantenerse y recuperarse de forma básica.

Resultado esperado: entorno operativo, documentado y mantenible.

13.3 Procedimiento resumido de despliegue y validación

Con el objetivo de facilitar una repetición ordenada del proyecto, se resume a continuación el flujo mínimo de despliegue:

1. Crear la instancia **Amazon EC2** en AWS.
2. Generar el par de claves y descargar el archivo `.pem`.
3. Configurar el grupo de seguridad con los puertos necesarios.
4. Asociar una **IP pública elástica** a la instancia.
5. Conectarse al servidor mediante **SSH**.
6. Instalar **Docker**, **Docker Compose**, **Ansible** y las utilidades base.
7. Clonar el repositorio del proyecto.
8. Revisar la estructura del repositorio y los ficheros de configuración.
9. Ejecutar `docker compose up -d`.
10. Verificar el estado de los contenedores con `docker ps`.
11. Comprobar el acceso a **Nginx**, **Prometheus**, **Uptime Kuma** y **cAdvisor**.
12. Validar **MariaDB** mediante creación, inserción, consulta y comprobación de persistencia.
13. Ejecutar los scripts de comprobación, copia de seguridad y restauración.
14. Registrar las incidencias detectadas y aplicar las correcciones necesarias.

Este procedimiento se siguió de forma progresiva durante el desarrollo del proyecto y constituyó la base operativa del despliegue final.

13.4 Registro de incidencias y troubleshooting

Durante el desarrollo y la implantación del proyecto se detectaron diversas incidencias técnicas. Su resolución permitió mejorar la estabilidad del entorno y reforzar la documentación operativa.

13.4.1 Incidencia 1. Error de autenticación al subir cambios a GitHub

Descripción: el repositorio remoto devolvía un error de acceso al intentar ejecutar git push.

Causa: autenticación incorrecta mediante HTTPS o falta de permisos en la clave configurada.

Corrección aplicada: se modificó el método de autenticación y se ajustó el acceso al repositorio.

Resultado: el proyecto quedó sincronizado correctamente con GitHub.

13.4.2 Incidencia 2. Permisos insuficientes sobre el socket de Docker

Descripción: el usuario no podía ejecutar comandos de Docker sin privilegios elevados.

Causa: el usuario todavía no pertenecía al grupo docker o la sesión no había actualizado los permisos.

Corrección aplicada: se añadió el usuario al grupo docker y se abrió una nueva sesión para aplicar los cambios.

Resultado: Docker y Docker Compose pudieron ejecutarse correctamente desde el usuario de trabajo.

13.4.3 Incidencia 3. Acceso incorrecto a MariaDB

Descripción: se produjo un error inicial al intentar acceder al contenedor utilizando un cliente o una sintaxis no válidos.

Causa: existía una diferencia entre el comando esperado y el cliente realmente disponible dentro del contenedor.

Corrección aplicada: se utilizó el cliente mariadb adecuado y se validaron las credenciales de acceso.

Resultado: se accedió correctamente a la base de datos y se ejecutaron las pruebas SQL previstas.

13.4.4 Incidencia 4. Validación de persistencia de MariaDB

Descripción: fue necesario comprobar que los datos se mantenían tras el reinicio del contenedor.

Causa: se debía verificar el funcionamiento real del volumen persistente asociado a MariaDB.

Corrección aplicada: se creó una tabla de prueba, se insertaron datos, se reinició el contenedor y se realizó una consulta posterior.

Resultado: la persistencia de datos quedó validada correctamente.

13.4.5 Incidencia 5. Ajuste de puertos y acceso externo

Descripción: algunos servicios requerían una revisión de las reglas de entrada para poder ser validados desde el navegador.

Causa: configuración inicial restrictiva del grupo de seguridad o intento de acceso desde una IP no autorizada.

Corrección aplicada: se revisó el **Security Group** y se ajustaron los puertos autorizados.

Resultado: los servicios publicados quedaron accesibles correctamente.

Análisis complementario de riesgos técnicos:

A continuación, se identificaron los riesgos técnicos principales y las medidas de mitigación aplicadas durante la implantación y las pruebas.

Riesgo técnico	Impacto	Probabilidad	Justificación breve	Mitigación y evidencia
Exposición de SSH o monitorización a redes no autorizadas	Alto	Media	Podría comprometer la instancia o exponer métricas internas. La probabilidad depende de errores en reglas de red.	Restricción a IP del administrador y solo puertos necesarios. Evidencia: Security Group y tabla 6.5.
Publicación accidental de credenciales	Alto	Baja	Una contraseña expuesta permitiría accesos no deseados. La probabilidad baja por .gitignore, .env.example y revisión de capturas.	Plantilla segura, exclusión del .env y capturas censuradas. Evidencia: apartado 6.3.
Pérdida de datos de MariaDB	Medio/alto	Media	Afectaría a la continuidad de la base de datos. No hay alta disponibilidad, pero sí volumen persistente y copias.	Volúmenes Docker, backup.sh y restore.sh. Evidencia: CP-08 a CP-11.
Errores por instalaciones manuales repetitivas	Medio	Alta	Los comandos manuales generan omisiones y diferencias entre ejecuciones. Retrasan la implantación.	Automatización con Bash y Ansible. Evidencia: instalar.sh, playbook.yml y CP-07.

Riesgo técnico	Impacto	Probabilidad	Justificación breve	Mitigación y evidencia
Dificultad para reproducir el despliegue	Medio	Media	Sin ficheros versionados sería más difícil repetir el entorno en otra instancia.	Repositorio GitHub y Docker Compose. Evidencia: CP-01 y CP-14.
Uso de componentes fuera del alcance	Medio	Media	Aumenta complejidad y mantenimiento sin aportar valor esencial.	Alcance acotado y descarte de Grafana o Portainer. Evidencia: diseño final.

Tabla 13.1 : Análisis complementario de riesgos técnicos y medidas de mitigación.

Fuente: elaboración propia.

13.5 Documentación complementaria del repositorio

El repositorio del proyecto centraliza la implementación real y mantiene la trazabilidad entre la memoria y el código fuente. En él se incluyeron los scripts Bash, el inventario y el playbook de Ansible, el fichero docker-compose.yml, los ficheros de configuración del entorno y la documentación técnica asociada.

La existencia de este repositorio aportó varias ventajas:

- Control de versiones de los cambios realizados.
- Posibilidad de recuperar versiones funcionales anteriores.
- Separación clara entre documentación y código.
- Facilidad de revisión por parte del profesorado.
- Reutilización futura del proyecto en nuevos entornos.

Así, la memoria queda respaldada por una implementación accesible, estructurada y verificable, no solo por una propuesta teórica.

13.6 Índice de anexos y documentación externa

Se incluyó un índice de documentación complementaria asociada al proyecto. Estos elementos ampliaron la memoria principal y facilitaron la revisión técnica y la reproducción del entorno.

Anexo o archivo	Contenido	Utilidad para la revisión
Manual básico de usuario	Acceso SSH, servicios disponibles, puesta en marcha, comprobación, copia y restauración.	Permite operar el entorno de forma autónoma.
Planificación operativa	Fases de preparación, despliegue, validación y mantenimiento.	Ayuda a repetir la implantación de forma ordenada.
Registro de incidencias	Problemas detectados, causa probable, corrección aplicada y resultado.	Demuestra resolución práctica y control de cambios.

Anexo o archivo	Contenido	Utilidad para la revisión
Repositorio GitHub	Scripts Bash, ficheros de Ansible, configuración Docker Compose, monitorización y documentación.	Aporta trazabilidad del código y evidencias de implementación.
Ficheros de configuración	docker-compose.yml, .env.example, prometheus.yml, inventory.ini y playbook.yml. El fichero .env real se mantiene como configuración local no versionada.	Permiten reproducir el despliegue de forma segura, sin exponer credenciales reales, y comprobar la coherencia técnica del entorno.
Capturas de validación	AWS, SSH, contenedores, servicios web, métricas, copia de seguridad y restauración.	Sirven como evidencia visual del caso práctico resuelto.

Tabla 13.2 : Índice de anexos y documentación externa.

Fuente: elaboración propia.

13.7 Cierre del apartado de anexos

Los anexos incluidos complementaron el contenido principal de la memoria y reforzaron el carácter práctico del proyecto. Gracias a ellos, se dispone de una referencia mínima para operar el entorno, repetir el despliegue, comprender el orden de implantación y consultar incidencias ya resueltas.

Su incorporación mejoró la mantenibilidad de la solución y aportó un valor adicional al proyecto, al dejar constancia no solo del resultado técnico obtenido, sino también del modo en que puede utilizarse, validarse y sostenerse en futuras iteraciones.